

UstaadAssist

The backend, explained from zero: every file, every route, every function.

Node.js · Express 5 · TypeScript · PostgreSQL · raw SQL

Semester planner API — Modules M1 to M7

Generated 27 September 2026

Contents

1 · What this backend is, in one page

- The problem the app solves
- What makes it not a CRUD app
- What the backend is responsible for
- The whole request flow, end to end
- The five layers, and why they are separate
- The technology, and why each piece
- The complete file map

2 · The syntax, explained

- TypeScript in ninety seconds
- Modern JavaScript used throughout
- Two Express-specific things that will bite

3 · The database

- How migrations work here
- The tables, grouped
- The three decisions worth defending
- Constraints — letting the database refuse bad data
- The SQL techniques used, from zero

4 · Authentication

- The thing to understand first
- The flow, with the actual values
- The part that is genuinely confusing: how the checking works
- `src/middleware/requireAuth.ts`, line by line
- `GET /auth/me` and the upsert that creates the teacher
- Ownership: the rule that makes the whole API safe

5 · The plumbing

- `src/http.ts` — one response shape, everywhere
- `src/middleware/error.ts` — the only place an error response is sent
- `src/db/pool.ts` — connections and transactions
- `src/index.ts` — the server, and why the order matters

6 · The planner — the graded core

- Why it is pure, and what "pure" means
- `slots.ts` — Section 3.1, which dates can class happen on
- `allocate.ts` — Section 3.2, spreading topics over slots
- `assessments.ts` — Section 3.3, a quiz can never come before its topics
- `replan.ts` — Section 3.4, freeze the past, rebuild the future
- `deficit.ts` — Section 3.5, the demo highlight
- `health.ts` — Section 3.6, how far behind are we
- The tests

7 · The grading engine

- Why it is separate from the PDF
- The three rules
- How the calculation actually runs
- What comes out
- The defaults a new course starts with
- A worked example from the seeded demo

8 · The services

The shape every service follows

`plan.service.ts` — load, think, save

`session.service.ts` — marking a class conducted or cancelled

`attendance.service.ts` — present by default

`student.service.ts` — the reviewed class list

`grading.service.ts` — marks in, results out

`course.service.ts` — settings, topic edits and cloning

`dashboard.service.ts` — every number in one response

`report.service.ts` — everything except the drawing

9 · Every endpoint

The rules that apply to all of them

Health and identity

Courses

Topics

Holidays

The planner

Sessions and attendance

Students

Assessments, marks and grading

Material, dashboard and reports

What is not in the contract

10 · What is stubbed, and why

The two stubs

Why 503 and not 501 or 500

The extraction stub

The PDF stub

11 · Seeding and smoke testing

`scripts/seed.ts` — a realistic full semester

`scripts/smoke.sh` — is every route actually wired up

The full verification, in order

12 · Quick reference

The five questions most likely to be asked

The five rules that must not be broken

The numbers

Where to look for what

Two things left open

Finally: the order things were built in

1 · What this backend is, in one page

Read this chapter and you will know what the server does and why it exists. Everything after it is detail.

The problem the app solves

A university teacher sets up a course once: the name, the start and end dates, the days they teach, and a list of topics. From that point the system **works out the teaching timetable itself** and **keeps it correct** as the semester falls apart in real life — classes get cancelled, public holidays land on a Monday, a topic that was supposed to take one class takes three.

Around that plan sit the everyday jobs: attendance, marks, grading, course material, reports and a dashboard.

What makes it not a CRUD app

A CRUD app is Create, Read, Update, Delete. A form saves a row, a list shows the rows. Most university projects are exactly that, and the requirement for this project was explicitly that it must **not** be.

The thing that makes UstaadAssist different is the **planner**. The planner is not storage — it is a calculation. You give it dates, class days, holidays and topics, and it computes a timetable. You tell it a class was cancelled and it recomputes the future while leaving the past alone. When there is not enough time left in the semester, it does not show an error: it computes three different ways out and lets the teacher choose.

That calculation is the graded core of the project. Chapter 6 covers it function by function, and it is the part you should be able to explain on a whiteboard without looking at the code.

What the backend is responsible for

The backend is a **REST API**. It has no screens. It speaks JSON over HTTP and the mobile app draws everything.

Its job is exactly four things:

- **Keep the data.** One PostgreSQL database with 16 tables.
- **Run the planner.** Take the course setup, compute the timetable, save it, and recompute it when things change.
- **Compute the results.** Take the marks and the weightage, compute every student's weighted total and letter grade, and produce the report data.
- **Check who is asking.** Every request must carry a valid token, and a teacher can only ever see their own courses.

And two things it is explicitly **not** responsible for:

- **It does not do sign-in.** Supabase Auth handles that inside the mobile app. The backend only **verifies** the token Supabase issued. Chapter 5 explains exactly how.
- **Files never pass through it.** The app uploads a photo or a PDF straight to Supabase Storage and sends the backend only the resulting path.

The whole request flow, end to end

Here is what happens when the teacher taps something in the app. This single picture explains the shape of every file in the project.



Every feature in this backend follows that same shape: **route** → **service** → **(planner or SQL)** → **response**. Once you have seen it once, the other thirty endpoints are variations.

The five layers, and why they are separate

Layer	Folder	Its one job	Does it touch the database?
routes/	src/routes/	Read the request, call a service, send the answer	No
services/	src/services/	Load data, call the planner, save the result	Yes — this is the only place that does

Layer	Folder	Its one job	Does it touch the database?
planner/	src/planner/	Compute the timetable. Pure calculation	No, never
grading/	src/grading/	Compute results and grades. Pure calculation	No, never
middleware/	src/middleware/	Run before or after every route: auth, errors	No

The important line in that table is the one that appears twice: **the planner and the grading engine never touch the database**. They take data as arguments and return data. Nothing else.

That is not a style preference. It buys three concrete things:

- **They can be tested.** A test can call `generateSlots("2026-09-01", "2026-12-18", ["mon", "wed"], [])` and check the answer, with no database running, in under a millisecond. There are 54 such tests.
- **They cannot surprise you.** The same inputs always produce the same output. There is no hidden state, no clock, no network.
- **They can be explained.** In a viva you can walk through `allocateTopics` on paper, because it is a loop over an array, not a query.

This is the rule most likely to be broken accidentally. If you ever find yourself wanting to add a `pool.query` inside `src/planner/`, that is the signal that the data should have been loaded by the service and passed in as an argument instead.

The technology, and why each piece

Piece	Version	Why it is here
Node.js	22	The JavaScript runtime. Version 22 matters — it can run TypeScript directly and it has a built-in test runner and <code>.env</code> loader
Express	5	The web framework. Handles routing and middleware
TypeScript	7	JavaScript with types. Catches mistakes before the code runs
PostgreSQL	17.6	The database, hosted on Supabase
pg	8.23	The PostgreSQL driver. Sends SQL, returns rows
jose	6.2	Verifies the signature on Supabase's tokens

That is the entire runtime dependency list. Six packages.

Three deliberate absences

No ORM. No Prisma, no Sequelize, no Knex. Every query in this project is SQL you can read. An ORM would have written the queries for you and hidden what the database is actually doing — and in a viva, "the library generated it" is not an answer. Writing the SQL also meant using the parts of PostgreSQL that actually fit the problem, like `unnest` to insert forty rows in one statement.

No build step. There is no `dist/` folder and nothing is compiled. Node 22 runs `.ts` files directly with the `--experimental-strip-types` flag, which throws the type annotations away at load time and runs the JavaScript underneath. `tsc --noEmit` is run separately, purely to check the types — it never produces output files.

No nodemon, no dotenv. Node 22 has `--watch` (restart when a file changes) and `--env-file` (load `.env`) built in. Two fewer packages to explain.

Running it

Command	What it does
<code>npm run dev</code>	Start the server and restart on every save
<code>npm start</code>	Start the server once
<code>npm test</code>	Run all 55 planner and grading tests
<code>npm run typecheck</code>	Check the types without running anything
<code>npm run seed</code>	Build a full realistic demo semester in the database
<code>npm run smoke</code>	Call all 33 endpoints once and print the status codes

The first time on a new machine, in order:

```
npm install

# create the tables
psql "$DATABASE_URL" -f migrations/001_init.sql
psql "$DATABASE_URL" -f migrations/002_makeup_date.sql

# create a login to test with
npm run create-demo-user

# fill the database with a realistic semester
npm run seed

npm run dev
```

The four secrets in .env

```
PORT=4000
DATABASE_URL=postgresql://...      <- full read/write on the database
SUPABASE_URL=https://xxx.supabase.co <- public, safe to ship in the app
SUPABASE_SERVICE_ROLE_KEY=eyJ...   <- full admin on the Supabase project
```

.env is in .gitignore. .env.example lists the key **names** with no values, so somebody cloning the repo knows what to fill in.

DATABASE_URL and SUPABASE_SERVICE_ROLE_KEY are both total-compromise secrets. Anyone holding either one can read or delete everything. Neither belongs anywhere near the React Native app. SUPABASE_URL and the **anon** key are the only two values that are safe in the app, and the anon key is not even needed by this backend.

The complete file map

Every file in the project, with one line on what it does. The chapters after this go through them properly.

The plumbing

File	What it does	Chapter
<code>src/index.ts</code>	Starts the server, mounts every router in order	5
<code>src/http.ts</code>	<code>ok()</code> , <code>AppError</code> , <code>pathId()</code> — the response shape	5
<code>src/db/pool.ts</code>	The database connection pool and <code>transaction()</code>	5
<code>src/middleware/requireAuth.ts</code>	Verifies the Supabase token	4
<code>src/middleware/error.ts</code>	The only place that sends an error response	5

The pure calculation — the graded core

File	What it does	Chapter
<code>src/planner/slots.ts</code>	Works out the real dates class can happen on	6
<code>src/planner/allocate.ts</code>	Spreads topics across those dates	6
<code>src/planner/assessments.ts</code>	Moves a quiz that falls before its topics	6
<code>src/planner/replan.ts</code>	Freezes the past, rebuilds the future	6
<code>src/planner/deficit.ts</code>	Computes the three ways out of a shortage	6
<code>src/planner/health.ts</code>	How many weeks behind the teacher is	6
<code>src/grading/compute.ts</code>	Weighted totals and letter grades	7

The services — the only code that touches the database

File	What it does	Chapter
<code>src/services/ownership.ts</code>	"Does this teacher own this row?"	8
<code>src/services/plan.service.ts</code>	Load / think / save for the planner	8
<code>src/services/session.service.ts</code>	Mark a class conducted or cancelled	8
<code>src/services/attendance.service.ts</code>	Present-by-default attendance, percentages	8
<code>src/services/student.service.ts</code>	Save a reviewed class list, student records	8
<code>src/services/grading.service.ts</code>	Assessments, marks, weightage, results	8
<code>src/services/course.service.ts</code>	Course settings, cloning, topic edits	8
<code>src/services/material.service.ts</code>	Course material records	8
<code>src/services/dashboard.service.ts</code>	Every dashboard number in one response	8
<code>src/services/report.service.ts</code>	Assembles the three reports	8
<code>src/services/extraction.service.ts</code>	Reads a class-list photo — stub	10

The routes

File	Mounted at	Chapter
<code>src/routes/auth.ts</code>	<code>/auth</code>	9
<code>src/routes/courses.ts</code>	<code>/courses</code>	9
<code>src/routes/plan.ts</code>	<code>/courses</code>	9
<code>src/routes/sessions.ts</code>	<code>/sessions</code>	9
<code>src/routes/topics.ts</code>	<code>/topics</code>	9
<code>src/routes/assessments.ts</code>	<code>/assessments</code>	9
<code>src/routes/students.ts</code>	<code>/students</code>	9
<code>src/routes/materials.ts</code>	<code>/materials</code>	9

Everything else

File	What it does
<code>src/data/pakHolidays.ts</code>	The Pakistani public holidays that get preloaded
<code>migrations/001_init.sql</code>	Creates 15 tables
<code>migrations/002_makeup_date.sql</code>	Creates the 16th, for agreed makeup classes
<code>tests/planner.test.ts</code>	20 tests for slots, allocation and quiz dates
<code>tests/replan.test.ts</code>	20 tests for replanning, deficit and health
<code>tests/grading.test.ts</code>	15 tests for result computation
<code>scripts/seed.ts</code>	Builds a realistic full-semester demo course
<code>scripts/create-demo-user.ts</code>	Creates a login through the Supabase Admin API
<code>scripts/smoke.sh</code>	Calls every endpoint once and checks the status code

2 · The syntax, explained

Every piece of TypeScript and JavaScript punctuation used in this project, with a real line from the codebase beside it. Skip this chapter if it is already familiar and come back when a symbol looks strange.

TypeScript in ninety seconds

TypeScript is JavaScript with labels on the data. You write what **kind** of value something is, and the compiler tells you when you have made a mistake before the program runs.

```
// JavaScript — nothing stops this
function addDays(date, count) { ... }
addDays(7, "2026-09-01")           // arguments in the wrong order. Runs. Breaks later.

// TypeScript — the mistake is caught immediately
function addDays(date: string, count: number) { ... }
addDays(7, "2026-09-01")           // error before it ever runs
```

The annotations are **erased** before the code runs. Node strips `: string` and `: number` out and runs plain JavaScript. The types exist only while you are writing.

type — giving a shape a name

```
export type Slot = {
  date: string;
  week_no: number;
};
```

That says: "a `Slot` is an object with a `date` which is text, and a `week_no` which is a number." Now `Slot[]` means "an array of those", and if you write `slot.dat` by mistake, you are told.

Used all over the planner, because the planner is mostly about moving well-defined shapes around.

Union types — "one of these"

```
status: "planned" | "conducted" | "cancelled";
```

The `|` means **or**. This field cannot hold `"Conducted"` or `"done"` or `"planed"` — only those three exact strings. This mirrors the `check` constraint in the database, so a typo is caught twice: once by TypeScript and once by PostgreSQL.

? on a property — optional

```
export type ReplanInput = {
  today: string;
  extraDates?: string[];    // may be left out entirely
};
```

`extraDates?` means the caller does not have to provide it. Inside the function its type is `string[] | undefined`, which is why the code immediately does `input.extraDates ?? []`.

Generics — `<T>`

```
export async function transaction<T>(fn: (client: PoolClient) => Promise<T>): Promise<T>
```

`<T>` is a placeholder for "whatever type the caller's function returns". If your function returns a course, `transaction` returns a course. If it returns a number, `transaction` returns a number. Without generics you would have to write `any` and lose all type checking through the transaction.

Read that signature aloud: "transaction takes a function, which is given a client and returns a promise of T, and transaction itself returns a promise of T."

`Record<K, V>` — an object used as a lookup

```
weightage: Record<string, number>;
```

"An object whose keys are strings and whose values are numbers", like `{ quiz: 15, final: 45 }`.

`Record` does not check that a key **exists**. `weightage["typo"]` type-checks fine and returns `undefined` at runtime. That is exactly why the grading code writes `weightage[component] ?? 0` instead of trusting it.

Modern JavaScript used throughout

`async / await` — waiting without callbacks

Talking to a database takes time. `await` says "pause here until the answer arrives, then carry on".

```
const result = await pool.query("select * from course where id = $1", [courseId]);
const course = result.rows[0];
```

Without `await`, `result` would be a **promise** — an IOU, not the rows — and `result.rows` would be `undefined`. A function containing `await` must be declared `async`.

The important consequence: an `async` function always returns a promise, so the caller must `await` it too. That is why almost every function in `services/` is `async` and every route handler is `async`.

`??` — the null-coalescing operator

"Use the left side, unless it is `null` or `undefined`, in which case use the right side."

```
const run = client ?? pool;           // use the transaction client if given, else the pool
const extraDates = input.extraDates ?? []; // an empty array if nothing was passed
```

It is different from `||` in a way that matters here:

```
0 || 10    // -> 10    ... wrong! zero is a legitimate value
0 ?? 10    // -> 0     ... correct
```

Since this codebase deals in marks and percentages, where `0` is a real answer, `??` is used and `||` is avoided.

`?.` — optional chaining

"Look inside this, but if it is missing, give me `undefined` instead of crashing."

```
const topicId = conducted.rows[0]?.topic_id;
```

If the update matched no rows, `rows[0]` is `undefined`. Plain `rows[0].topic_id` would throw **Cannot read properties of undefined**. With `?.` the whole expression is simply `undefined`, and the next line checks for that.

`!` — the non-null assertion

"I know TypeScript thinks this could be missing. It cannot be. Trust me."

```
ok(res, await listCourses(req.auth!.userId));
```

TypeScript declares `req.auth` as optional, because on most requests it is. But this route sits behind `requireAuth`, which either sets `req.auth` or throws — so by the time this line runs it is always there. The `!` says so.

`!` turns off a safety check. It is correct here because `requireAuth` guarantees it, and it would be a bug anywhere that guarantee does not hold. It is the one symbol in this project that deserves a second look every time you type it.

Destructuring — unpacking in one line

```
const { name, code, semester, start_date, end_date, class_days } = req.body;
```

Instead of six lines of `const name = req.body.name;`. It also works on function parameters:

```
const { today, startDate, endDate, classDays, holidays } = input;
```

Spread — ...

Two different jobs depending on where it appears.

Copying and overriding an object:

```
return { ...r, total_marks: Number(r.total_marks) };
```

"Everything from `r`, but with `total_marks` replaced by a number." The original `r` is untouched — a new object is made. Used constantly when converting what PostgreSQL returns into what the API sends.

Copying an array before sorting:

```
const ordered = [...topics].sort((a, b) => a.order_no - b.order_no);
```

`.sort()` sorts **in place** — it modifies the array you give it. In a pure function that would be a side effect on the caller's data. `[...topics]` makes a copy first, so the caller's array is left exactly as it was.

Array methods

These replace almost every `for` loop in the codebase.

Method	What it gives back	Example from the code
<code>.map()</code>	A new array, same length, each item transformed	<code>sessions.map((s) => s.date)</code>
<code>.filter()</code>	A new array with only the items that pass a test	<code>sessions.filter((s) => s.status === "conducted")</code>
<code>.find()</code>	The first matching item, or undefined	<code>remaining.find((r) => r.id === t.id)</code>
<code>.some()</code>	<code>true</code> if at least one item passes	<code>result.overflow.some((o) => o.id === t.id)</code>
<code>.reduce()</code>	One value built up from all of them	<code>totals.reduce((sum, t) => sum + t, 0)</code>
<code>.sort()</code>	The same array, reordered	<code>bands.sort((a, b) => b.min_percentage - a.min_percentage)</code>
<code>.flatMap()</code>	Like <code>.map()</code> , then flattened one level	<code>s.components.flatMap((c) => c.missing)</code>

`.reduce()` is the one worth reading slowly:

```
const needed = topicsRemaining.reduce((total, t) => total + t.sessions_needed, 0);
```

Start `total` at `0`. For each topic, add its `sessions_needed` to the running total. Give back the final total. It is a summing loop written as one expression.

Comparison functions in `.sort()`

`.sort()` needs to know which of two items comes first. You give it a function that returns a negative number, zero, or a positive number.

```
// smallest first
(a, b) => a.order_no - b.order_no

// largest first – just swap the operands
(a, b) => b.min_percentage - a.min_percentage
```

And for two levels of sorting, "if the first comparison is a tie, fall back to the second":

```
const candidates = [...topics].sort((a, b) => {
  const byPriority = PRIORITY_ORDER[a.priority] - PRIORITY_ORDER[b.priority];
  return byPriority !== 0 ? byPriority : b.order_no - a.order_no;
});
```

Lowest priority first; and between two topics of the same priority, the later one in the course goes first.

Map and Set

A plain object can be used as a lookup, but `Map` and `Set` are clearer and faster for this.

Set — a bag of unique values, for fast "is it in there?" checks:

```
const isHoliday = new Set(holidays);
...
if (teaches.has(dayName) && !isHoliday.has(date)) { ... }
```

Checking `.has()` on a `Set` is instant no matter how many holidays there are. `holidays.includes(date)` would scan the whole array each time, once per day of a four-month semester.

Map — key to value, where the key can be anything:

```
const markBy = new Map<string, GradingMark>();
for (const m of marks) {
  markBy.set(`${m.student_id}:${m.assessment_id}`, m);
}
// later, instantly:
const mark = markBy.get(`${student.id}:${assessment.id}`);
```

Without this, finding one student's mark for one assessment would mean searching the whole marks array — for 42 students times 8 assessments, that is 336 searches through hundreds of rows. The `Map` turns all of it into direct lookups.

Template literals — backticks

```
`${m.student_id}:${m.assessment_id}`
```

Backticks let you drop values into a string with `${...}`, and let the string span multiple lines. Every SQL query in this project is written in backticks so it can be laid out and indented like SQL rather than squashed onto one line.

`for...of` **versus** `for...in`

```
for (const topic of ordered) { ... }    // each ITEM <- what we use
for (const key in someObject) { ... }   // each KEY
```

`for...of` walks the values of an array. `for...in` walks the keys of an object. Mixing them up is a classic bug; this codebase uses `for...of` for arrays and `Object.entries()` for objects.

`Object.entries()` — looping over an object

```
Object.entries(results.summary.grade_distribution)
  .filter([, count] => count > 0)
  .map([grade, count] => `${grade}: ${count}`)
  .join(" ");
```

`Object.entries({A: 3, B: 5})` gives `[["A", 3], ["B", 5]]` — an array of key/value pairs, which the normal array methods can then work on. The `[grade, count]` in the arrow function is destructuring each pair. The `[, count]` in the filter skips the first element because it is not needed.

Two Express-specific things that will bite

An error handler is recognised by having exactly four parameters

```
// ORDINARY middleware – three parameters
export function notFoundHandler(req: Request, res: Response) { ... }

// ERROR handler – FOUR parameters
export function errorHandler(err, _req, res, _next) { ... }
```

Express decides which is which by counting the parameters. Delete the unused `_next` from the error handler and Express silently reclassifies it as ordinary middleware: it stops running on errors, and every failed request hangs forever with no response.

That is why `_next` is there and why it is named with an underscore — the underscore is a convention meaning "required by the signature, deliberately unused", and it also stops TypeScript complaining.

In Express 5, a thrown error in an `async` route is caught for you

```
router.get("/:courseId", requireAuth, async (req, res) => {
  ok(res, await getCourse(pathId(req.params, "courseId"), req.auth!.userId));
});
```

There is no `try/catch` anywhere in that route, and there is none in any route in this project. If `getCourse` throws — because the course does not exist, or the database is down — Express 5 catches it and hands it to `errorHandler`.

In Express 4 this was not true: an `async` throw would crash the process, and every route needed its own `try/catch` or a wrapper. Express 5 fixing this is why the routes in this codebase are three lines long.

This is the single most important reason the routes are readable. A route describes the happy path only. Every failure goes to one place.

`req.params` is not type-checked

```
req.params.courseId // TypeScript is perfectly happy. It is `undefined` at runtime.
```

TypeScript cannot know what parameter names a route declares, so `req.params` is an open bag of strings. A typo in the name compiles fine and then sends the literal string `"undefined"` to the database.

That is why every route in this project reads parameters through a helper:

```
pathId(req.params, "courseId")
```

which throws a clear 400 `courseId is missing from the URL` instead. Chapter 5 shows the implementation. This bug was hit twice while building the project, which is why the helper exists.

3 · The database

Sixteen tables, what each one holds, and why three of the design decisions are the way they are. Then the SQL techniques used, each explained from zero.

How migrations work here

There is no migration tool. A migration is a numbered .sql file in `migrations/`, run by hand:

```
psql "$DATABASE_URL" -f migrations/001_init.sql
psql "$DATABASE_URL" -f migrations/002_makeup_date.sql
```

Each file is wrapped in `begin; ... commit;` so it either applies completely or not at all. If a `create table` halfway down fails, the tables above it are rolled back too and you are left exactly where you started, rather than with half a schema.

Files are never edited after being applied. A change is a new numbered file.

The tables, grouped

Identity

Table	Holds
<code>teacher</code>	One row per signed-in teacher. Its <code>id</code> is the Supabase user id
<code>course</code>	Name, code, semester, start and end dates, class days, attendance threshold

Course setup

Table	Holds
<code>topic</code>	One row per topic: title, order, how many classes it needs, its minimum, priority, status
<code>holiday</code>	Dates class cannot happen. Preloaded, with an <code>is_active</code> flag the teacher toggles
<code>makeup_date</code>	Extra dates the teacher agreed to, outside the normal class days

The plan — the backbone of the app

Table	Holds
<code>session</code>	One row per class. Date, week number, which topic, which part of it, and its status
<code>replan_log</code>	A record of every replan: when, why, and what moved

People

Table	Holds
student	Roll number and name. Belongs to the teacher , not to one course
enrollment	Which student is in which course
attendance	One row per student per class: present, absent or leave

Assessment and marks

Table	Holds
assessment	A quiz, assignment, midterm, final or participation mark. Total marks, date
assessment_topic	Which topics an assessment covers — the planner needs this
mark	What one student scored on one assessment

Grading and material

Table	Holds
weightage	What percentage of the final result each component is worth
grade_scale	The letter grades and the minimum percentage for each
material	A pointer to a file in Supabase Storage. Never the file itself

`session` is the table everything else hangs off. Attendance is per session. Topic progress is derived from sessions. Every report counts sessions. Which is why the planner was built first: once sessions exist, half the app becomes straightforward.

The three decisions worth defending

1 · `mark.obtained` is nullable and has no default

```
obtained numeric(6,2),          -- NULL = not entered. Never 0 by default.
is_absent boolean not null default false,
```

There are three genuinely different situations, and they must not be confused:

Situation	<code>obtained</code>	<code>is_absent</code>	Counts as
Teacher has not typed it yet	NULL	false	Missing. Left out of the calculation
Student sat it and scored nothing	0	false	A real zero
Student did not sit it	NULL	true	A real zero

If `obtained` defaulted to 0, the first row would be indistinguishable from the second. A teacher halfway through entering the final exam would be looking at a screen full of students who appear to have failed.

This is the one schema detail most likely to get "tidied up" by someone adding `not null default 0` to make the code simpler. It would silently change grades. The tests in `tests/grading.test.ts` exist partly to catch that.

2 · A student belongs to the teacher, not to the course

```
create table student (
  teacher_id uuid not null references teacher(id) on delete cascade,
  roll_no    text not null,
  name       text not null,
  unique (teacher_id, roll_no)
);
```

The obvious design would put `course_id` on `student`. It was rejected because the same student takes several of the same teacher's courses. With `course_id` on the student you would get a duplicate row per course, and correcting a misread name would have to be done in each one.

Instead `student` holds the person once, and `enrollment` joins them to a course. The `unique (teacher_id, roll_no)` is what makes re-importing a class list safe: the second import matches the existing student rather than creating a twin.

3 · `session.status` drives everything

```
status text not null default 'planned'
check (status in ('planned', 'conducted', 'cancelled'))
```

Three values, and each one means something specific to the planner:

- `planned` — has not happened. The planner is free to delete and rebuild it.
- `conducted` — happened. **Never touched again.** This is "freeze the past".

- **cancelled** — was going to happen and did not. Kept as a record, not rebuilt, and deliberately **not** counted as a missed opportunity in the attendance percentage or the behind-schedule number.

That last point matters: if a cancelled class counted in the attendance denominator, students would lose attendance percentage because the **teacher** cancelled. The queries exclude cancelled classes for exactly that reason.

Constraints — letting the database refuse bad data

A **check** constraint is a rule PostgreSQL enforces itself. Even if a bug in the code tries to write nonsense, the database rejects it.

```
constraint course_dates_ck check (end_date > start_date),

constraint course_class_days_ck check (
  array_length(class_days, 1) between 1 and 7
  and class_days <@ array['mon','tue','wed','thu','fri','sat','sun']
),

constraint topic_min_le_needed_ck check (min_sessions <= sessions_needed)
```

Reading the second one: the class-days array must have between one and seven entries, and `<@` means "is contained by" — every value in `class_days` must appear in that list of seven day names. So `['mon', 'funday']` is rejected by the database.

There are 96 check constraints across the schema. They are the last line of defence, and they are checked **again** in the service code — not because the database check is unreliable, but because a raw constraint violation produces a message like `new row for relation "topic" violates check constraint "topic_min_le_needed_ck"`, and a teacher needs **"min_sessions (3) cannot be more than sessions_needed (2)"**.

unique — refusing duplicates

```
unique (course_id, order_no)      -- two topics cannot both be topic 5
unique (assessment_id, student_id) -- one mark per student per assessment
unique (session_id, student_id)  -- one attendance record per student per class
unique (teacher_id, roll_no)     -- one student row per roll number per teacher
```

A **composite** unique constraint like these means the combination must be unique, not each column on its own. Two different courses can both have a topic 5.

These are not just safety: the `on conflict upsert` pattern described below depends on them existing.

Foreign keys and `on delete`

```
course_id bigint not null references course(id) on delete cascade
topic_id  bigint          references topic(id)  on delete set null
```

`on delete cascade` — delete the course and every session, topic, holiday and enrollment goes with it. Correct for things that cannot exist without their parent.

`on delete set null` — delete a topic and the sessions survive with `topic_id` set to `NULL`. Correct here because a class that was **taught** still happened even if the topic record is later removed; losing the attendance attached to it would be worse than losing the topic label.

The SQL techniques used, from zero

The project uses raw SQL by choice, and it uses a handful of PostgreSQL features that are worth understanding properly because they appear repeatedly.

\$1, \$2 — parameters, and why they are not optional

```
await pool.query(
  "select id from course where id = $1 and teacher_id = $2",
  [courseId, req.auth!.userId],
);
```

The `$1` and `$2` are placeholders. The values are sent **separately** from the query text, so PostgreSQL never mixes the two up.

Compare the wrong way:

```
// NEVER do this
`select id from course where id = ${courseId}`
```

If `courseId` arrived from the app as `1; drop table course; --`, the second version hands PostgreSQL three statements and the table is gone. That is SQL injection. With `$1`, the value is only ever data — PostgreSQL treats it as the text `"1; drop table course; --"`, finds no course with that id, and returns nothing.

Every single query in this project uses parameters. There is no string interpolation of user input anywhere.

There is one place where a value is put into the query text directly: `update course set ${sets.join(", ")}`. Look closely and the `sets` array only ever contains strings the **code** built, like `"name = $2"` — the column names come from a fixed list in the code, and the values are still `$2`, `$3` and so on. User input never reaches the query text.

unnest — turning arrays into rows

This is the single most useful trick in the project. It is what makes it possible to insert forty rows with one statement.

The problem: the teacher pastes fourteen topic titles. The obvious code is a loop:

```
for (const title of titles) {
  await client.query("insert into topic (course_id, title) values ($1, $2)", [courseId, title]);
}
```

Fourteen separate round trips to a database in Mumbai. At 60ms each that is nearly a second of waiting.

`unnest` takes an array and turns it into rows:

```
select unnest(array['SQL', 'Joins', 'Normalization']);

unnest
-----
SQL
Joins
Normalization
```

So the whole insert becomes one statement:

```
insert into topic (course_id, order_no, title)
select $1, ordinality, title
  from unnest($2::text[]) with ordinality as t(title, ordinality)
```

Reading it piece by piece:

- `$2::text[]` — the second parameter, explicitly cast to an array of text. `pg` sends a JavaScript array as a PostgreSQL array, and the `::text[]` tells PostgreSQL which kind.
- `unnest(...)` — turn that array into one row per title.
- `with ordinality` — **also** give me the position number, 1, 2, 3. This is exactly what `order_no` needs, and it means the order the teacher typed is preserved without the code counting.
- `as t(title, ordinality)` — name the two resulting columns so the `select` above can use them.
- `select $1, ordinality, title` — build each row: the same course id every time, the position, the title.

One round trip. Fourteen rows.

Two arrays at once, which is how holidays are inserted with their names:

```
insert into holiday (course_id, date, name)
select $1, h.date, h.name
  from unnest($2::date[], $3::text[]) as h(date, name)
```

`unnest` with two arrays walks them in step: first date with first name, second with second. The code builds the two arrays with `.map()` and they line up because `.map()` preserves order.

The biggest one in the project unnests six arrays at once, to write a whole semester of sessions:

```
insert into session (course_id, topic_id, date, week_no, part_no, total_parts, kind)
select $1, t.topic_id, t.date, t.week_no, t.part_no, t.total_parts, t.kind
  from unnest($2::bigint[], $3::date[], $4::int[], $5::int[], $6::int[], $7::text[])
  as t(topic_id, date, week_no, part_no, total_parts, kind)
```

Thirty sessions, one statement.

on conflict — insert, or update if it is already there

Also called an **upsert**. The teacher opens the attendance screen a second time and fixes a mistake; the row already exists.

```
insert into attendance (session_id, student_id, status)
select ...
on conflict (session_id, student_id) do update
  set status = excluded.status,
      marked_at = now()
```

- `on conflict (session_id, student_id)` — "if inserting would break the unique constraint on these two columns".
- `do update set ...` — "then update the existing row instead of failing".
- `excluded` — a special name meaning **the row I was trying to insert**. So `excluded.status` is the new status.

The alternative is a `select` to check, then either an `insert` or an `update` — three statements, and a race if two requests arrive at once. `on conflict` is one statement and the database handles the race.

`do nothing` is the other option, used where an existing row should be left exactly as it is:

```
insert into holiday (course_id, date, name)
select ...
on conflict (course_id, date) do nothing
```

When the semester is extended, this tops up any newly-exposed public holidays without resetting the ones the teacher already ticked or unticked.

update ... from unnest(...) — many different updates in one statement

The holiday screen sends back a list of { id, is_active }. Every row may have a different value, so a single `set is_active = $1` will not do.

```
update holiday h
  set is_active = v.is_active
  from unnest($2::bigint[], $3::boolean[]) as v(id, is_active)
 where h.id = v.id and h.course_id = $1
```

`unnest` builds a temporary two-column table of id/value pairs, and the `where` joins each holiday to its own new value. Eight different updates, one round trip.

The `and h.course_id = $1` is not decoration — without it, someone could send holiday ids belonging to another teacher's course and change them.

count(*) filter (where ...) — several counts in one pass

```
select count(*) filter (where status = 'conducted') as conducted,
       count(*) filter (where status = 'cancelled') as cancelled,
       count(*) filter (where status = 'planned')   as planned
  from session where course_id = $1
```

Without `filter` this would be three separate queries, each scanning the table. With it, PostgreSQL walks the rows once and keeps three counters.

Used all over the dashboard and the attendance summary.

CTEs — with ... as (...)

A **common table expression** names a result so the rest of the query can use it, which keeps a complicated query readable.

```
with conducted as (
  select id from session where course_id = $1 and status = 'conducted'
)
select s.roll_no,
       count(a.id) filter (where a.status = 'present') as present
  from enrollment e
  join student s on s.id = e.student_id
  left join attendance a on a.student_id = s.id
                        and a.session_id in (select id from conducted)
 where e.course_id = $1
  group by s.id, s.roll_no
```

`conducted` is defined once at the top and referred to below. It reads as: "the classes that actually happened; now for each enrolled student, count their present marks among those classes."

left join versus join

This distinction decides whether a student with no marks appears at all.

- `join` — only rows that have a match on both sides.
- `left join` — every row from the left side, with `NULLS` where the right side has no match.

```
from enrollment e
join student s on s.id = e.student_id      -- every enrollment HAS a student
left join mark m on m.student_id = s.id    -- a student may have NO marks yet
      and m.assessment_id = $1
```

The marks-entry screen needs **every** student in the class, including the ones not yet marked — otherwise the teacher could not enter their mark. So `mark` is joined with `left join`, and the code reads `m.obtained` as `null` for those students.

Using a plain `join` there would quietly hide exactly the students the teacher still has work to do on.

`coalesce` — a fallback for `NULL`

```
coalesce(a.status, 'present') as status
coalesce(array_agg(at.topic_id) filter (where at.topic_id is not null), '{}') as topic_ids
```

`coalesce(x, y)` gives `x` unless it is `NULL`, in which case `y`.

The first line implements present-by-default on the attendance screen: a student with no attendance row shows as present.

The second is defensive: `array_agg` over no rows returns `NULL`, not an empty array, and `NULL` would crash the JavaScript that maps over it. `'{}'` is an empty PostgreSQL array.

`to_char(date, 'YYYY-MM-DD')` — dates as plain text

```
select to_char(start_date, 'YYYY-MM-DD') as start_date
```

Every date is read out of the database as a formatted string rather than a date type.

The reason is timezones. `pg` converts a PostgreSQL `date` into a JavaScript `Date`, which is a moment in time in **your** timezone. A course starting `2026-09-01` becomes midnight local, and depending on the server's timezone that can print as 31 August. The planner then generates a slot on the wrong day.

Turning it into the string `"2026-09-01"` before it leaves the database removes the problem entirely. The planner works in strings throughout, and string comparison on `YYYY-MM-DD` sorts correctly by date — which is why the code can write `if (s.date <= today)` and be right.

This pairs with the planner using `Date.UTC` internally. Both halves of the system refuse to let a local timezone anywhere near a calendar date. Chapter 6 covers the planner side.

`numeric` and `Number()`

PostgreSQL `numeric` is an exact decimal type — the right choice for marks and percentages, because floating point cannot represent `0.1` exactly and grades should not drift.

`pg` returns `numeric` as a **string**, precisely so no precision is lost on the way out. That is why service code is full of:

```
total_marks: Number(r.total_marks),  
obtained: r.obtained === null ? null : Number(r.obtained),
```

The second line matters: `Number(null)` is `0`, so a plain `Number(r.obtained)` would turn every un-entered mark into a zero — the exact bug the nullable column was designed to prevent. The explicit `null` check keeps it `null`.

4 · Authentication

The backend does not have a login screen, a password field, or a users table it controls. It only ever **checks** something the app already has. This chapter explains what that something is and why checking it is safe.

The thing to understand first

There are two completely separate jobs hiding inside the word "login":

- **Authentication** — proving who you are. Email, password, "forgot password", email confirmation, sessions.
- **Verification** — being handed a claim of identity and deciding whether to believe it.

Supabase Auth does the first. This backend does only the second. CLAUDE.md is explicit: **do not build authentication**.

The flow, with the actual values

1. TEACHER SIGNS IN — inside the mobile app, talking to Supabase.
The password never touches your Express server.

```
app ---- email + password ----> Supabase Auth
app <---- access_token ----- Supabase Auth
```

The access_token is a JWT. It looks like this:

```
eyJhbGciOiJIUzI1NiIsImtpZCI6IjZWNjZTA1... . eyJzdWIiOiJkYTFmMTh... . MEQCIgX...
\_____ header _____/ \_____ payload _____/ \_ signature _/
```

2. APP CALLS YOUR BACKEND, carrying the token.

```
GET /courses
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

3. YOUR BACKEND VERIFIES IT — without asking Supabase anything.

```
requireAuth
- splits off the "Bearer " prefix
- fetches Supabase's PUBLIC key (once, then cached)
- checks the signature with that public key
- checks it has not expired
- sets req.auth = { userId: "da1f18ab-...", email: "..."} }
```

4. EVERY QUERY IS SCOPED TO THAT userId.

```
select * from course where id = $1 and teacher_id = $2
                                     ^^ from the token
```

The part that is genuinely confusing: how the checking works

The natural assumption is that step 3 must call Supabase to ask "is this token real?". It does not. It never contacts Supabase to check a token. Here is why that is safe.

Two keys, and only one of them is secret

Supabase holds a **private key**. Your backend gets the matching **public key**. They are a pair with one useful property:

- Only the private key can **create** a valid signature.
- The public key can **check** a signature, but cannot create one.

So the public key being public is fine. It is a lock that anyone may inspect but only one key opens.

The analogy that actually fits

Think of a wax seal on a letter. The king has a signet ring — one of a kind, and he never lends it. Everyone in the kingdom has **seen** the seal and knows what it looks like, so anyone can look at a sealed letter and say "yes, that is the king's seal". Knowing what the seal looks like does not let you make one, because you do not have the ring.

The private key is the ring. The public key is knowing what the seal looks like. The signature is the wax impression.

The question "if I give the backend the picture of the seal, can't anyone forge it?" is the right question to ask, and the answer is no — and this is not a trick of secrecy, it is mathematics. Verifying a signature and producing one are genuinely different operations, and only one of them needs the private key.

What the signature is actually over

The signature is computed from the header **and the payload**. So it is not a stamp that says "Supabase approves"; it is a stamp that says "Supabase approves ***exactly these contents***".

Change one character of the payload — swap your user id for somebody else's — and the signature no longer matches the contents. Verification fails. And you cannot compute the **new** correct signature for your altered payload, because that needs the private key.

This was tested directly while building the project: flipping a single character in the payload made `jose` reject the token immediately.

Where the public key comes from

Supabase publishes it at a fixed URL:

```
https://<your-project>.supabase.co/auth/v1/.well-known/jwks.json
```

JWKS means **JSON Web Key Set** — a list of public keys, each with an id. The real response for this project:

```
{
  "keys": [
    {
      "kty": "EC",
      "alg": "ES256",
      "kid": "23ecce05-b586-4f07-b8d3-fb2ae8d6e5f2",
      "crv": "P-256",
      "x": "...", "y": "..."
    }
  ]
}
```

<- elliptic curve
<- the signing algorithm
<- key id
<- the public key itself

The token's header names which key signed it:

```
{ "alg": "ES256", "kid": "23ecce05-b586-4f07-b8d3-fb2ae8d6e5f2" }
```

So `jose` reads the `kid`, picks that key out of the set, and verifies with it. The `kid` exists so Supabase can rotate keys: it publishes a new one, starts signing with it, and old tokens still verify against the old key until they expire.

The older Supabase approach used a single **shared secret** — the same string on both sides, used to both sign and verify. CLAUDE.md forbids it here: **"Do not verify with a shared JWT secret — the project uses asymmetric signing keys."** With a shared secret, your backend holds a key capable of **minting** tokens, so a leak from your server lets an attacker forge any user. With the public key, a leak gives away nothing.

src/middleware/requireAuth.ts, line by line

```
import { createRemoteJWKSet, jwtVerify } from "jose";
import { unauthorized } from "../http.ts";
import type { NextFunction, Request, Response } from "express";
```

Extending the Request type

```
declare global {
  namespace Express {
    interface Request {
      auth?: { userId: string; email: string };
    }
  }
}
```

Express's `Request` type knows nothing about `req.auth` — that is ours. This block **adds** the property to Express's own type, so every route in the project can write `req.auth!.userId` and have TypeScript accept it.

`auth?` is optional because on a request that has not been through this middleware, it genuinely is not there. That optionality is what forces routes to write `req.auth!` — and that `!` is honest, because the route sits behind `requireAuth`.

The key set is created once

```
const JWKS = createRemoteJWKSet(
  new URL(`${process.env.SUPABASE_URL}/auth/v1/.well-known/jwks.json`),
);
```

This is at the **top** of the module, not inside the function, and that placement is the whole point. `createRemoteJWKSet` returns an object that fetches the keys on first use and then **caches** them.

Put it inside the handler and every single request would make an HTTPS call to Supabase before doing anything else — slow, and it would break entirely if Supabase were briefly unreachable. At module level it is created once when the server starts, and thousands of requests reuse the cached key.

The middleware itself

```
export async function requireAuth(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization;

  if (!header?.startsWith("Bearer ")) {
    throw unauthorized("Missing or malformed Authorization header");
  }

  const token = header.slice("Bearer ".length);

  try {
    const { payload } = await jwtVerify(token, JWKS);

    req.auth = {
      userId: payload.sub as string,
      email: (payload.email as string) ?? "",
    };
  } catch {
    throw unauthorized("Invalid or expired token");
  }

  next();
}
```

Walking through it:

- `req.headers.authorization` — the header, or `undefined` if absent.
- `!header?.startsWith("Bearer ")` — the `?`. handles the absent case without crashing. Covers a missing header and a malformed one in one condition.
- `header.slice("Bearer ".length)` — cuts off the seven characters `Bearer` and leaves the token. Written as `"Bearer ".length` rather than `7` so it says what it means.
- `await jwtVerify(token, JWKS)` — this is the whole verification. It checks the signature against the right key, checks `exp` has not passed, and throws if anything is wrong.
- `payload.sub` — `sub` is short for **subject**: the user's uuid. This is the value that becomes `teacher_id` on every row.
- `catch { throw unauthorized(...) }` — the real reason is deliberately not passed on. Telling a caller **"expired at 14:02"** versus **"signature invalid"** helps someone probing the API. One message covers both.
- `next()` — hand control to the next middleware, which is the route. Forgetting this leaves the request hanging forever with no response and no error.

Why the `catch` is empty-braced

```
} catch {
```

Modern JavaScript allows `catch` with no parameter when the error is not used. Here it genuinely is not: any failure means the same thing to the caller.

GET /auth/me and the upsert that creates the teacher

`src/routes/auth.ts` is short but does something worth explaining.

```
insert into teacher (id, email) values ($1, $2)
on conflict (id) do update set email = excluded.email
returning id, email, full_name, department, created_at
```

The situation: a teacher signs up through Supabase. Supabase creates a row in `auth.users`. Your `teacher` table knows nothing about it — there is no webhook, and the backend was not involved.

So the first time that teacher calls any endpoint, their `teacher` row has to appear. `/auth/me` is the natural place, because the app calls it right after sign-in.

- `insert ... on conflict (id) do update` — create the row, or if it exists, refresh the email. No "check then insert", no race.
- `set email = excluded.email` — keeps the email current if they changed it in Supabase.
- `returning ...` — hands back the row in the same statement, so there is no second `select`.

This matters because of the foreign key:

```
create table teacher (
  id uuid primary key references auth.users(id) on delete cascade,
  ...
);
```

`teacher.id` is the Supabase user id, and PostgreSQL enforces that it exists in `auth.users`. So a `teacher` row can only be created for a genuine Supabase user, and deleting the user in Supabase deletes the teacher and cascades to all their courses.

Ownership: the rule that makes the whole API safe

Verifying the token answers **who is calling**. It does not stop that person asking for somebody else's data. `GET /courses/47` is a perfectly well-formed request from a perfectly valid user — course 47 just is not theirs.

The rule in this project is absolute: **every query that takes an id from the URL also filters on the teacher id from the token.**

```
select ... from course
where id = $1 and teacher_id = $2
--                ^^ from req.auth, never from the body
```

`src/services/ownership.ts` centralises this so no route has to remember:

```
export async function assertCourseOwned(courseId: string, teacherId: string): Promise<CourseRow> {
  const result = await pool.query<CourseRow>(`
    select ... from course where id = $1 and teacher_id = $2`,
    [courseId, teacherId],
  );

  const course = result.rows[0];
  if (!course) throw notFound("Course");
  return course;
}
```

Every service function starts with a call to this or one of its siblings.

Why it answers 404 and not 403

A course that exists but belongs to someone else returns **404 Course not found**, not **403 Forbidden**.

403 would be more literally accurate, and that is exactly the problem: it confirms the id is real. Someone could walk `/courses/1`, `/courses/2`, `/courses/3` and learn how many courses exist and which ids are taken, purely from which error came back. 404 for both cases tells them nothing.

Rows two joins away

For `PATCH /sessions/:id` the URL carries a session, and the teacher is two tables away. `ownership.ts` handles it with a join:

```
select x.course_id
  from session x
    join course c on c.id = x.course_id
 where x.id = $1 and c.teacher_id = $2
```

If the session belongs to another teacher, the join produces no row, and the code throws `notFound("Session")`. The same helper is reused for topics, assessments and materials.

The other half of the rule: the owner is **never** read from the request body. If the app sent `{ "teacher_id": "..."}` it would be ignored. Anyone can put anything in a body; only the token is signed.

5 · The plumbing

Four small files that every other file depends on: the response shape, the error handling, the database pool, and the server itself.

`src/http.ts` — one response shape, everywhere

The problem it solves

Without a convention, different endpoints end up answering differently. One returns `{ courses: [...] }`, another returns a bare array, a third returns `{ data: ..., error: null }`. The app then needs different unwrapping code per endpoint, and three developers building three screens each guess differently.

So **every** response from this backend, success or failure, has exactly three keys:

```
{ "success": true,  "data": { ... }, "message": null }
{ "success": false, "data": null,    "message": "Course not found" }
```

The app can therefore be written once: check `success`, use `data`, or show `message`.

`ok()`

```
export function ok(res: Response, data: unknown, httpStatus = 200): void {
  res.status(httpStatus).json({ success: true, data, message: null });
}
```

- `data: unknown` — deliberately not `any`. `unknown` means "some value, I am not claiming to know what", and TypeScript will not let you accidentally use it as something specific. `any` would switch type checking off.
- `httpStatus = 200` — a default. Routes that create something pass `201` explicitly.
- `{ success: true, data, message: null }` — the bare `data` is shorthand for `data: data`.

Forgetting to call `ok()` is the classic bug and produces no error at all. The route finishes, Express never sends a response, and the phone sits spinning until it times out. This happened during development on `POST /courses`. Nothing catches it — no type error, no crash — so the habit to build is: every route path ends in either `ok()` or a `throw`.

`AppError` — an error that knows its status code

```
export class AppError extends Error {
  status: number;
  constructor(status: number, message: string) {
    super(message);
    this.name = "AppError";
    this.status = status;
  }
}
```

A normal `Error` carries only a message. The error handler needs to know whether this is a 404, a 400 or a 500, and whether the message is safe to show the user. `AppError` carries both facts.

`extends Error` means it is still a real error — `throw` works, stack traces work, `instanceof Error` is true. `super(message)` calls the parent constructor to set the message.

The three shortcuts on top:

```
export const notFound = (what: string) => new AppError(404, `${what} not found`);
export const badRequest = (message: string) => new AppError(400, message);
export const unauthorized = (message = "Unauthorized") => new AppError(401, message);
```

So a service writes `throw notFound("Course")` and the caller gets `404 { "message": "Course not found" }`.

`pathId()` — reading an id out of the URL safely

```
export function pathId(
  params: Record<string, string | string[] | undefined>,
  name: string,
): string {
  const value = params[name];
  if (typeof value === "string" && value !== "") return value;
  throw new AppError(400, `${name} is missing from the URL`);
}
```

It exists for two reasons.

First, the types. Express 5 types a route parameter as `string | string[]`, because a pattern could capture a name more than once. Ours never do, so this narrows it to `string` in one place instead of every route doing a cast.

Second, and more importantly, typos. `req.params` has no fixed set of keys, so TypeScript cannot catch this:

```
// route declares :courseId
const id = req.params.courseId; // compiles. Is `undefined`.
```

That `undefined` then reaches the database as the literal text `"undefined"`, which produces a confusing error far from the mistake. This exact bug was hit twice while building the project. `pathId(req.params, "courseId")` instead throws `"courseId is missing from the URL"` — which names the mistake.

src/middleware/error.ts — the only place an error response is sent

Two functions, and the order they are mounted in `index.ts` is what makes them work.

notFoundHandler — nothing matched the URL

```
export function notFoundHandler(req: Request, res: Response): void {
  res.status(404).json({
    success: false,
    data: null,
    message: `No route for ${req.method} ${req.path}`,
  });
}
```

Mounted **after every route**. Express tries each route in order; if none matched, control reaches here.

Without it, Express sends its own default 404, which is an HTML page. The app would call `response.json()` on a chunk of HTML and crash with something like **Unexpected token '<'**, which tells the mobile developer nothing about the real problem — a URL typo.

The obvious objection: **"this is a phone app, the user cannot type a URL, so how can a route be missing?"** The user cannot — but the **developer** can. A screen with `/courses/3/attendance/summary` misspelled, or a route renamed on the backend, produces exactly this. Having it answer in JSON means the mobile developer sees **"No route for GET /courses/3/attendance/summary"** in their console and fixes it in ten seconds.

errorHandler — something threw

```
export function errorHandler(
  err: unknown,
  _req: Request,
  res: Response,
  _next: NextFunction,
): void {
```

Four parameters. This is how Express recognises an error handler. Remove `_next` and it becomes ordinary middleware that never runs on an error — and every failure hangs.

It handles four kinds of error, in order.

1 · Our own errors

```
if (err instanceof AppError) {
  res.status(err.status).json({ success: false, data: null, message: err.message });
  return;
}
```

The status and message came from our code, so both are safe to send. This is the path for `notFound`, `badRequest` and `unauthorized`.

2 · A body that is not valid JSON

```
if (err instanceof SyntaxError && (err as { type?: string }).type === "entity.parse.failed") {
  res.status(400).json({ success: false, data: null, message: "The request body is not valid JSON" });
  return;
}
```

`express.json()` throws this when it cannot parse the body. It is the **client's** mistake, so a 400 with a clear message is right.

This case was found by the smoke test: a malformed body was returning 500 `Something went wrong`, which would have had the mobile team hunting a server bug over a stray comma.

3 · Database errors that are really the caller's fault

```
function describeDatabaseError(code: string) {
  switch (code) {
    case "23505": return { status: 409, message: "That record already exists" };
    case "23503": return { status: 400, message: "That refers to something that does not exist" };
    case "23514": return { status: 400, message: "One of the values is not allowed" };
    case "22P02": return { status: 400, message: "One of the values is the wrong type" };
    default: return null;
  }
}
```

Those five-character codes are PostgreSQL's standard SQLSTATE codes: unique violation, foreign key violation, check violation, and bad input syntax. All four mean the request asked for something impossible, so a 500 would be misleading.

Anything not in that list falls through to the next case — an unexpected database error is treated as a real bug, not smoothed over.

4 · Everything else

```
console.error("[error]", err);

res.status(500).json({ success: false, data: null, message: "Something went wrong" });
```

Two different audiences, deliberately:

- **The terminal** gets the real error with its stack trace, because that is what fixes the bug.
- **The client** gets a generic sentence, because an internal error message can leak table names, column names, file paths and query text. That is information an attacker uses.

Why controllers never send errors themselves

Because Express 5 forwards anything an `async` route throws straight here. A route only describes success:

```
router.get("/:courseId", requireAuth, async (req, res) => {
  ok(res, await getCourse(pathId(req.params, "courseId"), req.auth!.userId));
});
```

One line. No `try`, no `catch`, no `if (!course) return res.status(404)...` Every failure — a bad id, a missing course, a dead database — lands in one function. Change the error format once and all thirty-plus endpoints change with it.

src/db/pool.ts — connections and transactions

What a pool is, and why not a single connection

Opening a connection to PostgreSQL is expensive: a TCP handshake, TLS, then authentication. Doing it per request would add well over a hundred milliseconds to every call to a database in Mumbai.

A **pool** opens a handful of connections when the server starts and lends them out. A query borrows one, uses it, and returns it.

```
export const pool = new Pool({ connectionString: process.env.DATABASE_URL });
```

The error listener that stops the process dying

```
pool.on("error", (err) => console.error("[db]", err));
```

This looks like a throwaway line. It is not.

A pooled connection can fail while sitting **idle** — the network drops, or Supabase restarts. The pool then emits an **'error'** event. In Node, an **'error'** event with **no listener** is not ignored: it is thrown as an uncaught exception and **the whole process exits**.

So without this one line, a brief network blip takes the entire server down. With it, the failure is logged, the pool discards the dead connection, and the next request gets a fresh one. Nothing else in the app notices.

This was demonstrated directly while building: a Node script emitting **'error'** with no listener died instantly, and adding an empty listener kept it alive. The listener is what matters, not what it does.

transaction() — all or nothing

The scenario

Creating a course does two things: insert the course, then insert its preloaded holidays. If the second one fails, you must not be left with a course that has no holidays — the teacher would open the setup screen, see no public holidays, and generate a plan that schedules classes on Eid.

Either both happen or neither does.

Why it must be written by hand

PostgreSQL has transactions, with **begin** and **commit**. The problem is **which connection** the commands run on.

`pool.query()` borrows a connection, runs one statement, and gives it straight back. So:

```
await pool.query("begin");    // borrows connection A, starts a transaction, returns A
await pool.query("insert ..."); // might borrow connection B – outside the transaction!
await pool.query("commit");   // might borrow connection C – commits nothing
```

This was verified during development by printing the PostgreSQL backend process id: five `pool.query()` calls landed on **different** backends. A transaction spread across three connections is not a transaction.

So the transaction must run on **one borrowed connection**, held for its whole duration. That is what this function does.

```
export async function transaction<T>(fn: (client: PoolClient) => Promise<T>): Promise<T> {
  const client = await pool.connect();

  try {
    await client.query("begin");
    const result = await fn(client);
    await client.query("commit");
    return result;
  } catch (err) {
    await client.query("rollback");
    throw err;
  } finally {
    client.release();
  }
}
```

Line by line:

- `pool.connect()` — borrow one connection and **keep** it. Unlike `pool.query()`, it is not returned automatically.
- `client.query("begin")` — start the transaction on that specific connection.
- `await fn(client)` — run the caller's work, **handing it the client**. Everything the caller does with that client is inside the transaction.
- `client.query("commit")` — make it permanent.
- `catch` — anything threw: `rollback` undoes every statement since `begin`, then `throw err` re-throws so the route still fails properly. Swallowing the error here would make a failed write look like a success.
- `finally` — `release()` gives the connection back to the pool. `finally` runs on success **and** failure, which is the point: a connection never returned is a connection permanently lost from the pool. Enough of those and the server stops responding.

Who gives the caller its client?

This confused during development and is worth stating plainly: `transaction` **does**. The caller writes a function that **accepts** a client, and `transaction` calls that function with the one it borrowed.

```
const course = await transaction(async (client) => {
  //          ^^^^^ transaction passes this in
  const created = await client.query("insert into course ...");
  await client.query("insert into holiday ...");
  return created.rows[0];
});
```

The caller never creates a client. It writes a recipe that says "given a client, here is what to do with it", and `transaction` supplies one, wraps it in `begin/commit`, and cleans up.

The <T> in the signature

```
transaction<T>(fn: (client: PoolClient) => Promise<T>): Promise<T>
```

Whatever the caller's function returns, `transaction` returns. Return a course row and you get a course row, fully typed. Without `<T>` the return type would have to be `any` and every call site would lose type checking.

Inside a transaction, use `client.query`, never `pool.query`. A stray `pool.query` borrows a **different** connection, runs outside the transaction, and is not rolled back. That is why the load helpers in `plan.service.ts` all take an optional client: `const run = client ?? pool;`

src/index.ts — the server, and why the order matters

```
const app = express();

app.use(express.json());

app.get("/health", (_req, res) => { ok(res, { status: "ok" }); });

app.use("/auth", authRoutes);

app.use("/courses", planRoutes);
app.use("/courses", courseRoutes);

app.use("/sessions", sessionRoutes);
app.use("/topics", topicRoutes);
app.use("/assessments", assessmentRoutes);
app.use("/students", studentRoutes);
app.use("/materials", materialRoutes);

app.use(notFoundHandler);
app.use(errorHandler);

app.listen(PORT, () => { console.log(`Listening on http://localhost:${PORT}`); });
```

Express walks this list top to bottom for every request. Reading it in order **is** reading the request lifecycle.

app.use() **versus** app.get()

- `app.get("/health", ...)` — runs only for GET `/health`.
- `app.use(express.json())` — runs for **every** request, whatever the method or path.
- `app.use("/courses", courseRoutes)` — runs for every request whose path starts with `/courses`, and strips that prefix before handing over. So inside `courses.ts`, `router.get("/")` means GET `/courses`.

`express.json()` **must be first**

It reads the request body and parses it. A route that runs before it would find `req.body` as `undefined`.

`/health` **touches nothing**

```
app.get("/health", (_req, res) => { ok(res, { status: "ok" }); });
```

Deliberately no database query. It answers "is the server process alive and able to respond". If it also queried the database, a database problem would make the server look dead when it is not — two different failures reporting as one.

Two routers on `/courses`

```
app.use("/courses", planRoutes);
app.use("/courses", courseRoutes);
```

Express allows several routers on the same prefix; it tries them in order. The planner is mounted first so it reads as the most important part of the file, which it is.

It would work either way: Express matches the **full path**, so `courseRoutes`' plain `/:courseId` route does not swallow `/:courseId/plan/generate` — those are different paths. But putting the planner first makes the

intent obvious to the next person.

The two handlers at the bottom are the safety net

They must be **last**. `notFoundHandler` only makes sense once every route has had its chance, and `errorHandler` is registered last so Express knows it is the final destination for anything thrown anywhere above.

Move either one above the routes and it stops working — `notFoundHandler` would answer 404 for everything.

```
process.env.PORT ?? 4000
```

```
const PORT = Number(process.env.PORT ?? 4000);
```

Use the port from `.env`, or 4000 if it is not set. `Number()` because environment variables are always strings and `app.listen("4000")` is not the same as `app.listen(4000)`.

6 · The planner — the graded core

Six files, no database access in any of them, 40 tests. This is the part that makes the project more than a management system, and the part to be able to explain on a whiteboard.

Why it is pure, and what "pure" means

A **pure** function has two properties: the same inputs always give the same output, and it changes nothing outside itself. No queries, no file writes, no reading the clock.

Every function in `src/planner/` is pure. The service loads the data, calls the planner, and writes the result back. The planner itself only calculates.

The clock is an argument, not a fact

```
export type ReplanInput = {
  today: string;      // "2026-11-18"
  ...
};
```

`today` is **passed in**. The planner never calls `new Date()`.

This is the single decision that makes the whole thing testable. A test can say "pretend it is 18 November, five classes have been conducted, two were cancelled — what does the plan look like?" and get an answer instantly. If the planner read the clock, that test could only be run on one day of the year.

It also makes the demo possible: the seed script and the dashboard endpoint both accept `today`, so a whole semester can be shown off in September.

Dates are strings in YYYY-MM-DD

The planner never handles a JavaScript `Date` across a function boundary. Inside `slots.ts`, dates are converted to `Date` objects fixed at midnight **UTC**, and converted straight back:

```
function parseDate(iso: string): Date {
  return new Date(`${iso}T00:00:00Z`);    // the Z forces UTC
}

function toISODate(d: Date): string {
  return d.toISOString().slice(0, 10);
}
```

And every calculation uses the UTC methods — `getUTCDate()`, `setUTCDate()` — never `getDay()` or `setDate()`.

Without the Z, `new Date("2026-09-01")` is interpreted in the **local** timezone. On a machine set to UTC+5, midnight local is 19:00 the previous day in UTC, so `getDay()` can report the wrong weekday and the planner generates a slot on a day the teacher does not teach. This class of bug is invisible in testing if your machine happens to be in UTC, and appears only in production. Using UTC throughout removes the possibility.

A useful side effect: `"2026-09-01" < "2026-11-18"` is true as a string comparison, because YYYY-MM-DD sorts alphabetically in date order. That is why the code can write `sessions.filter((s) => s.date <=`

today) and be correct.

slots.ts — Section 3.1, which dates can class happen on

A **slot** is one real date a class can be held: it falls on a day the teacher teaches, and it is not a holiday.

```
export function generateSlots(
  startDate: string,
  endDate: string,
  classDays: string[],
  holidays: string[],
): Slot[] {
  const teaches = new Set(classDays);
  const isHoliday = new Set(holidays);

  const start = parseDate(startDate);
  const end = parseDate(endDate);

  const slots: Slot[] = [];
  const cursor = new Date(start);

  while (cursor.getTime() <= end.getTime()) {
    const date = toISODate(cursor);
    const dayName = DAY_NAMES[cursor.getUTCDay()]!;

    if (teaches.has(dayName) && !isHoliday.has(date)) {
      const daysIn = Math.floor((cursor.getTime() - start.getTime()) / MS_PER_DAY);
      slots.push({ date, week_no: Math.floor(daysIn / 7) + 1 });
    }

    cursor.setUTCDate(cursor.getUTCDate() + 1);
  }

  return slots;
}
```

The algorithm is deliberately boring: walk one day at a time from the start date to the end date, and keep the ones that qualify.

- `new Set(classDays) / new Set(holidays)` — built once, outside the loop. Over a four-month semester the loop runs about 110 times; `Set.has()` is instant, whereas `holidays.includes()` would scan the array each time.
- `DAY_NAMES[cursor.getUTCDay()]` — `getUTCDay()` returns 0 for Sunday through 6 for Saturday, so `DAY_NAMES` is deliberately ordered `["sun", "mon", "tue", "wed", "thu", "fri", "sat"]` to line up with it.
- The week number — days since the start, divided by seven, plus one. So the first seven days are week 1. This is why `week_no` is relative to the course, not the calendar year.
- `cursor.setUTCDate(cursor.getUTCDate() + 1)` — the standard way to add a day. It handles month ends and leap years by itself: `setUTCDate(32)` on a 31-day month rolls into the 1st of the next month.

A worked example. Course from Monday 1 September to Sunday 20 September 2026, teaching Monday and Wednesday, with 2 September a holiday:

Date	Weekday	Teaches?	Holiday?	Slot
2026-09-01	mon	yes	no	week 1
2026-09-02	tue	no	—	—

Date	Weekday	Teaches?	Holiday?	Slot
2026-09-03	wed	yes	no	week 1
2026-09-07	mon	yes	no	week 2
2026-09-09	wed	yes	no	week 2
2026-09-14	mon	yes	no	week 3
2026-09-16	wed	yes	no	week 3

suggestMakeupDates — finding free days

Used by the "Extend" deficit option. It looks for days the teacher does **not** normally teach:

```
const isNormalClassDay = teaches.has(dayName);
const isSunday = dayName === "sun";

if (!isNormalClassDay && !isSunday && !isHoliday.has(date)) {
  found.push(date);
}
```

Sundays are excluded because a makeup class on a Sunday is not a realistic suggestion in this context. Holidays are excluded for the obvious reason. `cursor.setUTCDate(cursor.getUTCDate() + 1)` before the loop makes it **strictly** after the given date, so "after today" does not include today.

mergeExtraDates — folding makeup classes back in

Once the teacher has agreed to a makeup date, it has to become a usable slot. It cannot come out of `generateSlots`, because by definition it is not a normal class day.

```
export function mergeExtraDates(slots: Slot[], extraDates: string[], startDate: string): Slot[] {
  if (extraDates.length === 0) return slots;

  const already = new Set(slots.map((s) => s.date));
  const start = parseDate(startDate);

  const extras: Slot[] = [];
  for (const date of extraDates) {
    if (already.has(date)) continue;
    already.add(date);

    const daysIn = Math.floor((parseDate(date).getTime() - start.getTime()) / MS_PER_DAY);
    extras.push({ date, week_no: Math.floor(daysIn / 7) + 1 });
  }

  return [...slots, ...extras].sort((a, b) => a.date.localeCompare(b.date));
}
```

Two details that matter:

- `already.add(date)` inside the loop, not just the check at the top — so the same makeup date appearing twice in the input only produces one slot.
- The final `.sort()` — the makeup date must land in **date order**, not be appended at the end. Otherwise the next topic allocated would be scheduled on the makeup date before the earlier regular classes, and the week-by-week screen would be out of sequence.

allocate.ts — Section 3.2, spreading topics over slots

```
export function allocateTopics(slots: Slot[], topics: PlannerTopic[]): Allocation {
  const ordered = [...topics].sort((a, b) => a.order_no - b.order_no);

  const sessions: PlannedSession[] = [];
  const overflow: PlannerTopic[] = [];

  let next = 0; // index of the next free slot

  for (const topic of ordered) {
    const need = Math.max(1, topic.sessions_needed);

    if (next + need > slots.length) {
      overflow.push(topic);
      continue;
    }

    const spansSeveral = need > 1;

    for (let part = 1; part <= need; part++) {
      const slot = slots[next]!;
      next += 1;

      sessions.push({
        date: slot.date,
        week_no: slot.week_no,
        topic_id: topic.id,
        part_no: spansSeveral ? part : null,
        total_parts: spansSeveral ? need : null,
      });
    }
  }

  return { sessions, overflow, unusedSlots: slots.slice(next) };
}
```

The idea is one pointer walking through the slot list. Each topic consumes as many slots as it needs, and the pointer moves on.

- `[...topics].sort(...)` — a **copy** is sorted, so the caller's array is untouched. Sorting by `order_no` rather than trusting the input order means the service does not have to guarantee it.
- `Math.max(1, topic.sessions_needed)` — defensive. A topic needing zero classes would consume nothing and never be taught; treating it as 1 is the sane interpretation.
- `part_no` and `total_parts` are `null` for a single-class topic — so the app can show **"Normalization (2 of 3)"** only where it makes sense, rather than **"Introduction (1 of 1)"**.

The one real design decision: a topic is never half-placed

```
if (next + need > slots.length) {
  overflow.push(topic);
  continue;
}
```

If a topic needs three classes and only two slots remain, the **whole topic** goes to overflow. It does not get two classes with the third missing.

The reasoning: two-thirds of Normalization taught and then abandoned is worse than the teacher being told clearly **"Normalization does not fit; here are your three options"**. Half-teaching hides the problem; overflow surfaces it, and surfacing it is what the deficit screen is for.

Note also `continue` rather than `break` — later topics are still tried. A short topic after a long one may well still fit, and dropping everything after the first failure would be needlessly pessimistic.

`calculateDeficit`

```
export function calculateDeficit(slotsAvailable: number, topicsRemaining: PlannerTopic[]): number {
  const needed = topicsRemaining.reduce((total, t) => total + Math.max(1, t.sessions_needed), 0);
  return needed - slotsAvailable;
}
```

Straight from the spec: `deficit = sessions_still_needed - slots_still_available`. Zero or less means everything fits.

assessments.ts — Section 3.3, a quiz can never come before its topics

This is the rule the spec calls non-optional: **a quiz must never be scheduled before its topics are taught.**

The logic:

1. For each assessment, look at the topics it covers. 2. Find the **latest** date any of those topics is taught. 3. If the assessment is scheduled earlier than that, move it to the next available slot after that date. 4. Return a sentence the teacher can read.

The sentence is the part that matters for the demo:

```
Quiz 3 moved from 19 Oct to 11 Nov because Normalization
will not be completed before 4 Nov.
```

Three pieces of information: what moved, where to, and **why**. A plan that silently rearranges itself is untrustworthy; one that explains itself is a feature.

Details worth knowing:

- An assessment with **no date** or **no topics** is left alone. A final exam covering everything, or an assessment the teacher has not dated yet, should not be moved by this rule. The test `an assessment with no date or no topics is ignored` covers it.
- An assessment covering several topics waits for the **last** of them, not the first.
- An assessment already scheduled after its topics is left exactly where it is.
- The move target is **the next slot**, not "the last topic date plus one day" — a quiz has to land on a day there is a class.

The service then saves the move:

```
update assessment
  set date = $2,
    original_date = coalesce(original_date, $3),
    move_reason = $4
where id = $1
```

`coalesce(original_date, $3)` keeps the **first** original date through repeated replans. If Quiz 3 was set for 19 Oct, moved to 11 Nov, then moved again to 18 Nov, `original_date` stays 19 Oct — which is what the teacher recognises.

replan.ts — Section 3.4, freeze the past, rebuild the future

Triggered when a class is cancelled or a topic's `sessions_needed` changes mid-semester.

The principle

- A class marked `conducted` is **never touched**.
- Dates from today onward are worked out again.
- Only teaching that has not happened yet is re-allocated.
- The result reports **what changed**, not just the new plan.

```
export function replan(input: ReplanInput): ReplanResult {
  const { today, startDate, endDate, classDays, holidays, existingSessions, topics } = input;
  const extraDates = input.extraDates ?? [];

  const frozen = existingSessions.filter((s) => s.status === "conducted");
  const oldPlanned = existingSessions.filter((s) => s.status === "planned");

  const taken = new Set(frozen.map((s) => s.date));
  const futureFrom = today > startDate ? today : startDate;

  const regular = generateSlots(futureFrom, endDate, classDays, holidays);
  const futureExtras = extraDates.filter((d) => d >= futureFrom && d <= endDate);

  const slots: Slot[] = mergeExtraDates(regular, futureExtras, startDate).filter(
    (s) => !taken.has(s.date),
  );

  const stillToTeach = remainingWork(topics, frozen);
  const { sessions, overflow } = allocateTopics(slots, stillToTeach);

  return {
    frozen,
    sessions,
    overflow,
    changes: describeChanges(oldPlanned, sessions, topics),
    slotsAvailable: slots.length,
    deficit: calculateDeficit(slots.length, stillToTeach),
  };
}
```

Reading the important lines:

- `futureFrom = today > startDate ? today : startDate` — if the semester has not started yet, rebuild from the start date rather than from today. Without this, replanning a future course would throw away the first weeks.
- `taken` and the `.filter((s) => !taken.has(s.date))` — a date that already holds a conducted class is removed from the available slots, so the rebuild cannot schedule a second class on top of a class that already happened.
- `extraDates.filter((d) => d >= futureFrom ...)` — a makeup date in the past is not a future slot.

`remainingWork` — how much of each topic is still owed

```
function remainingWork(topics: PlannerTopic[], conducted: ExistingSession[]): PlannerTopic[] {
  const taught = new Map<number, number>();
  for (const s of conducted) {
    if (s.topic_id === null) continue;
    taught.set(s.topic_id, (taught.get(s.topic_id) ?? 0) + 1);
  }

  const still: PlannerTopic[] = [];
  for (const t of topics) {
    const done = taught.get(t.id) ?? 0;
    const left = Math.max(1, t.sessions_needed) - done;
    if (left > 0) still.push({ ...t, sessions_needed: left });
  }
  return still;
}
```

This is the heart of "rebuild the future". A topic needing three classes with two already conducted still needs **one**, so it is pushed with `sessions_needed: 1`. A topic fully taught is not pushed at all.

`taught.set(s.topic_id, (taught.get(s.topic_id) ?? 0) + 1)` is the standard counting idiom: get the current count or zero, add one, put it back.

`{ ...t, sessions_needed: left }` makes a **copy** with a different value — the caller's topic objects are not modified. Purity again.

describeChanges — the "what moved" screen

The spec is explicit that the response must report what changed, not just the new plan. This function compares the old planned sessions with the new ones.

```
const group = <T extends { topic_id: number | null; date: string }>(rows: T[]) => {
  const byTopic = new Map<number, string[]>();
  for (const r of rows) {
    if (r.topic_id === null) continue;
    const list = byTopic.get(r.topic_id) ?? [];
    list.push(r.date);
    byTopic.set(r.topic_id, list);
  }
  for (const list of byTopic.values()) list.sort();
  return byTopic;
};
```

Group both the before and after lists by topic, with each topic's dates sorted. Then walk them side by side:

```
const longest = Math.max(oldDates.length, newDates.length);
for (let i = 0; i < longest; i++) {
  const from = oldDates[i];
  const to = newDates[i];

  if (from && to && from !== to) {
    changes.push({ kind: "moved", topic_id: topicId, topic: name, from, to });
  } else if (!from && to) {
    changes.push({ kind: "added", topic_id: topicId, topic: name, to });
  } else if (from && !to) {
    changes.push({ kind: "removed", topic_id: topicId, topic: name, from });
  }
}
```

Matching the **first** future class of a topic before against the **first** after, the second against the second, and so on. Three outcomes: it moved, it is new, or it is gone.

`Math.max(oldDates.length, newDates.length)` so neither list is cut short — a topic that gained a class shows an **added**, one that lost a class shows a **removed**.

The output is exactly what the app needs:

```
{ kind: "moved", topic_id: 2, topic: "Joins", from: "2026-09-14", to: "2026-09-21" }
```

deficit.ts — Section 3.5, the demo highlight

When there is not enough time left, this is the response. **It does not throw an error.** It computes three ways out and lets the teacher choose.

Option 1 — DROP: cut whole topics

```
const candidates = [...topics].sort((a, b) => {
  const byPriority = PRIORITY_ORDER[a.priority] - PRIORITY_ORDER[b.priority];
  return byPriority !== 0 ? byPriority : b.order_no - a.order_no;
});
```

Two-level sort. Lowest priority first; and between two topics of equal priority, the **later** one in the course goes first — because earlier topics tend to be the foundations the later ones build on. Dropping "Introduction to Databases" to save "NoSQL" would be the wrong way round.

Then take topics until the deficit is covered:

```
for (const t of candidates) {
  if (recovered >= deficit) break;
  const freed = Math.max(1, t.sessions_needed);
  chosen.push({ topic_id: t.id, title: t.title, sessions_freed: freed });
  recovered += freed;
}
```

A **high** priority topic is only ever reached if dropping every **low** and **normal** one was not enough.

Option 2 — COMPRESS: shorten topics that have room

```
const withSlack = topics
  .map((t) => ({
    topic: t,
    slack: Math.max(1, t.sessions_needed) - Math.max(1, t.min_sessions),
  }))
  .filter((x) => x.slack > 0)
  .sort((a, b) => b.slack - a.slack);
```

Slack is how many classes a topic could lose before hitting its minimum. Topics with none are filtered out. The rest are sorted by most slack first, so the fewest topics are affected.

```
const take = Math.min(slack, deficit - recovered);
const from = Math.max(1, topic.sessions_needed);
chosen.push({ topic_id: topic.id, title: topic.title, from, to: from - take });
```

`Math.min(slack, deficit - recovered)` takes only what is needed — no point shrinking a topic by three classes when two would do. And `min_sessions` is never breached, because `take` can never exceed `slack`.

Option 3 — EXTEND: add classes on days off

```
function buildExtend(deficit: number, candidates: string[]): ExtendOption {
  const dates = candidates.slice(0, deficit);
  return {
    type: "extend",
    sessions_recovered: dates.length,
    covers_deficit: dates.length >= deficit,
    dates,
  };
}
```

Take exactly as many free dates from `suggestMakeupDates` as the deficit requires.

`covers_deficit` — the honesty flag

Every option carries it:


```
covers_deficit: recovered >= deficit
```

Because an option might not be enough. If the remaining topics are all at their minimum, compressing recovers nothing. If the semester is nearly over, there may be no free dates left. Rather than pretend, each option states plainly whether it actually solves the problem. The test `compress` reports honestly when it cannot cover the whole deficit exists for this.

`applyDeficitOption` — putting the choice into effect

```
if (choice === "drop") {
  const dropped = new Set(options.drop.topics.map((t) => t.topic_id));
  return { topics: topics.filter((t) => !dropped.has(t.id)), droppedTopicIds: [...dropped], extraDates: [] };
}

if (choice === "compress") {
  const newLength = new Map(options.compress.topics.map((t) => [t.topic_id, t.to]));
  return {
    topics: topics.map((t) => newLength.has(t.id) ? { ...t, sessions_needed: newLength.get(t.id)! } : t),
    droppedTopicIds: [], extraDates: [],
  };
}

return { topics, droppedTopicIds: [], extraDates: options.extend.dates };
```

Still pure — it returns the **adjusted data**, it does not save anything. The service takes that result, writes it to the database, and runs the planner again.

health.ts — Section 3.6, how far behind are we

```
export function scheduleHealth(input: HealthInput): Health {
  const { today, classesPerWeek, sessions, totalTopics, completedTopics } = input;

  const plannedUpToToday = sessions.filter(
    (s) => s.date <= today && s.status !== "cancelled",
  ).length;

  const conducted = sessions.filter((s) => s.status === "conducted").length;
  const cancelled = sessions.filter((s) => s.status === "cancelled").length;

  const perWeek = Math.max(1, classesPerWeek);
  const behind = (plannedUpToToday - conducted) / perWeek;

  return {
    planned_up_to_today: plannedUpToToday,
    conducted,
    cancelled,
    behind_by_weeks: Math.round(behind * 10) / 10,
    syllabus_percent: totalTopics === 0 ? 0 : Math.round((completedTopics / totalTopics) * 100),
    total_sessions: sessions.length,
  };
}
```

The formula from the spec:

```
behind_by_weeks = (sessions_planned_up_to_today - sessions_conducted) / classes_per_week
```

Three details:

- `s.status !== "cancelled"` in the first filter. **A cancelled class was never an opportunity.** If a cancelled class counted as "should have happened", the teacher would be reported as behind for a class the university closed. The test `a cancelled class does not count as being behind` locks this in.
- `Math.round(behind * 10) / 10` — one decimal place. Multiply by ten, round to a whole number, divide by ten. `1.4666...` becomes `1.5`.
- `Math.max(1, classesPerWeek)` — guards against dividing by zero, which would produce `Infinity` on the dashboard.

The dashboard then turns the number into the sentence:

```
warning: health.behind_by_weeks >= 0.5
? `You are ${health.behind_by_weeks} week${health.behind_by_weeks === 1 ? "" : "s"} behind your plan.`
: null
```

Below half a week it says nothing, because being a single class behind is normal and a permanent warning would be ignored.

The tests

`npm test` runs 55 tests across three files, in about 300 milliseconds, with no database.

The five the spec requires, and where they are:

Required case	Test name
Holidays removed from slots	holidays are removed from the slots
Multi-session topic spanning classes	a multi-session topic spans consecutive classes and is numbered
Quiz auto-moving when topics slip	a quiz moves when its topics will not be finished in time
Replanning leaving conducted sessions untouched	a conducted class is never touched
Deficit producing three valid options	a deficit always produces three options

And the edge cases that were added because they are the ones that actually break:

- a semester that crosses into the next year still works — the day-walking loop across December into January.
- a topic is never half-placed — the overflow rule.
- compress never takes a topic below its minimum.
- compress reports honestly when it cannot cover the whole deficit.
- an agreed makeup date becomes a usable class slot — and that it lands in date order.
- a makeup date in the past is not offered as a future slot.
- a cancelled class does not count as being behind.

After changing anything in `src/planner/`, run `npm test` before doing anything else. It takes a third of a second and it is the only thing standing between a small refactor and a silently wrong timetable.

7 · The grading engine

The second pure calculation. It takes marks and weightage and produces every student's weighted total and letter grade. `src/grading/compute.ts`, 15 tests.

Why it is separate from the PDF

The spec is blunt about this: **"Computation is a pure function, kept separate from PDF rendering. Never calculate inside the PDF code."**

The reason is a specific failure it prevents. If the results **screen** computed the totals and the result **PDF** computed them too, there would be two implementations of the same arithmetic. They would agree at first and then drift — someone fixes a rounding bug in one and forgets the other. Then a student sees 76.5% on the phone and 76% on the printed sheet, and the whole thing loses credibility.

Here there is one function. The screen calls it. The PDF calls it. They cannot disagree, because there is nothing to disagree about.

```

      computeResults()      <- one function, pure
      /                    \
GET /courses/:id/results  buildResultReport()
      (the screen)          (the PDF data)

```

The three rules

Rule 1 — round once, at the end

```

function round2(value: number): number {
  return Math.round(value * 100) / 100;
}

```

Called in exactly one place per student: after the weighted sum is complete.

Why it matters. Suppose three components each score 33.3333...% with weights adding to 100.

Approach	Working	Result
Round each component first	33.33 + 33.33 + 33.34, each already rounded	drifts
Round once at the end	33.3333... summed, then rounded	exact

The drift is small — usually well under a percent. But grade boundaries are exact numbers. A student on 84.996% should be an A- (the boundary is 85). Accumulated rounding can push them to 85.0 and hand them an A they did not earn, or the reverse.

The test a `borderline total is not pushed over a grade boundary by early rounding` uses 5 out of 6 = 83.333...%, and checks the result is `83.33` and the grade is `A-`, not `A`.

Component percentages **are** rounded, but only for display — `components.map((c) => ({ ...c, percentage: round2(c.percentage) })))` happens **after** `weightedSum` has already been calculated from the unrounded values. The rounded numbers are shown; the exact ones are used.

Rule 2 — a missing mark is not a zero

```
const mark = markBy.get(`${student.id}:${assessment.id}`);

if (mark === undefined || (mark.obtained === null && !mark.is_absent)) {
  missing.push({ assessment_id: assessment.id, title: assessment.title });
  missingCount += 1;
  continue;
}
```

`continue` — the assessment is skipped entirely. It is left out of **both** the numerator and the denominator.

So a student with 8/10 on Quiz 1 and nothing entered for Quiz 2 has a quiz percentage of **80%** (8 out of 10), not 40% (8 out of 20).

The alternative — treating it as zero — would mean that halfway through entering the final exam, every student not yet reached appears to be failing. A teacher glancing at the dashboard would see a catastrophe that is really just unfinished data entry.

And the student is **flagged**, not quietly adjusted:

```
has_missing_marks: missingCount > 0,
missing_count: missingCount,
```

The result PDF names them explicitly:

The totals below are provisional for the students listed here, because at least one mark has not been entered. A missing mark is not counted as zero.

CT-21007 Hira Malik: mark not entered for Quiz 4.

Rule 3 — an absent student is a zero

```
obtained += mark.is_absent ? 0 : (mark.obtained ?? 0);
outOf += assessment.total_marks;
```

Note that `outOf` increases **either way**. An absent student scores zero out of the full marks — the paper counts against them.

This is the opposite of Rule 2 and the distinction is the whole point. Three different situations:

Situation	obtained	is_absent	Numerator	Denominator	Flagged missing?
Not entered yet	null	false	—	—	yes
Sat it, scored 0	0	false	+0	+total	no
Did not sit it	null	true	+0	+total	no

The test `absent` and `not-entered` give different results for the same student proves rows two and three of that table produce 50% and 100% respectively for the same student — which is exactly right, because in one case the teacher has recorded a fact and in the other they have not finished typing.

How the calculation actually runs

Setting up two lookups

```
const markBy = new Map<string, GradingMark>();
for (const m of marks) {
  markBy.set(`${m.student_id}:${m.assessment_id}`, m);
}

const byComponent = new Map<Component, GradingAssessment[]>();
for (const a of assessments) {
  const list = byComponent.get(a.type) ?? [];
  list.push(a);
  byComponent.set(a.type, list);
}
```

The first maps "12:34" — student 12, assessment 34 — to that mark. The second groups assessments by type, so all the quizzes are in one list.

Without these, finding one student's mark would mean scanning the whole marks array. For 42 students times 8 assessments that is 336 scans through 300-odd rows. With the `Map` it is 336 instant lookups.

The per-student loop

For each student, for each of the five components:

```
for (const component of COMPONENTS) {
  const weight = weightage[component] ?? 0;
  const list = byComponent.get(component) ?? [];

  let obtained = 0;
  let outOf = 0;
  const missing: ComponentResult["missing"] = [];

  for (const assessment of list) {
    // ... the three rules above
  }

  const percentage = outOf === 0 ? null : (obtained / outOf) * 100;
  const contribution = percentage === null ? 0 : (percentage * weight) / 100;

  weightedSum += contribution;
  ...
}
```

- `weightage[component] ?? 0` — a component the teacher gave no weight contributes nothing.
- `outOf === 0 ? null : ...` — no assessments of this type, or all of them missing. `null`, not `0` — because "no quizzes exist" is not the same as "scored zero on the quizzes". Dividing by zero would give `NaN`, which would spread through every subsequent number.
- `contribution` — the component's percentage scaled by its weight. 85% on a component worth 20 contributes 17 points.

Quizzes are summed, not averaged

This is a real decision and it is easy to get wrong. If a student scores 5/5 on Quiz 1 and 10/20 on Quiz 2:

Method	Working	Result
Sum the marks (what we do)	15 out of 25	60%
Average the percentages	$(100\% + 50\%) / 2$	75%

Summing is correct because a 20-mark quiz should count four times as much as a 5-mark one. Averaging the percentages treats them as equally important, which silently inflates the score of anyone who did well on a small paper.

The test `quizzes are added together, not averaged separately` uses exactly those numbers and asserts 60.

The grade

```
export function gradeFor(percentage: number, scale: GradeBand[]): string {
  const bands = [...scale].sort((a, b) => b.min_percentage - a.min_percentage);

  for (const band of bands) {
    if (percentage >= band.min_percentage) return band.grade;
  }

  return bands[bands.length - 1]?.grade ?? "-";
}
```

Sort highest first, then return the first band the percentage reaches.

- `[...scale].sort(...)` — a copy, and the sort means the scale can arrive in **any** order and still work. The test `a scale given in the wrong order still works` covers it.
- `percentage >= band.min_percentage` — the boundary is inclusive. Exactly 85 is an A.
- The fallback after the loop is unreachable in practice, because the lowest band starts at 0. It is there so the function always returns a string rather than `undefined`.

The scale is never hardcoded

`DEFAULT_GRADE_SCALE` is a ten-band scale seeded per course the first time it is asked for. The teacher can then replace it entirely, and `PUT /courses/:id/grade-scale` validates:

- every band needs a letter and a percentage between 0 and 100
- no duplicate letters
- the lowest band must start at 0, or a very low mark would have no grade at all

The test `the grade scale is not hardcoded` runs the same 82% mark through the default scale and a three-band scale, and gets `A-` and `A`. Different universities grade differently, so hardcoding would have made the app unusable outside one institution.

What comes out

```
export type StudentResult = {
  student_id: number;
  roll_no: string;
  name: string;
  components: ComponentResult[];
  weighted_total: number;
  grade: string;
  has_missing_marks: boolean;
  missing_count: number;
};
```

And the class summary:

```
export type ClassSummary = {
  student_count: number;
  class_average: number | null;
  highest: number | null;
  lowest: number | null;
  grade_distribution: Record<string, number>;
  pass_count: number;
  fail_count: number;
  students_with_missing_marks: number;
};
```

| `null` on the averages is not laziness — an empty class has no average, and `0` would be a lie. The test an empty class does not crash the summary checks all three come back `null`.

`isPass` treats the **lowest band on the scale** as the failing grade, whatever it is called. So a scale using `E` instead of `F` still counts correctly, without the code knowing anything about the letter `F`.

The defaults a new course starts with

```
export const DEFAULT_WEIGHTAGE: Record<Component, number> = {
  quiz: 15, assignment: 10, midterm: 25, final: 45, participation: 5,
};
```

Totals 100. Seeded the first time `GET /courses/:id/weightage` is called, not at course creation — which is the minimum-input rule from the spec: **"Students, weightage and material are asked for later, only when the feature that needs them is opened."**

The teacher opens the grading screen and finds it already filled in with something sensible, and corrects it. They are never shown an empty form at setup time.

`PUT /courses/:id/weightage` enforces the total:

```
if (Math.abs(total - 100) > 0.01) {
  throw badRequest(`The weightage must total 100. It currently totals ${total}.`);
}
```

The `0.01` tolerance exists so `33.33 + 33.33 + 33.34` is accepted. Requiring exactly 100 with decimal inputs would reject legitimate splits.

A worked example from the seeded demo

One student, with the default weightage, partway through the semester:

Component	Weight	Marks	Percentage	Contribution
Quizzes	15%	24 / 30	80.00%	12.00
Assignments	10%	31 / 40	77.50%	7.75
Midterm	25%	38 / 50	76.00%	19.00
Final	45%	not sat	—	0.00
Participation	5%	none set	—	0.00
			Weighted total	38.75

38.75% and a grade of F — and both are **correct**, because half the grade (the 45% final plus 5% participation) has not happened yet. The `has_missing_marks` flag is what tells the screen and the PDF to print this as provisional.

This is worth understanding before the demo: a mid-semester result sheet **should** show low totals and failing grades. The number is not the story — the missing-mark flag is. Once the final is entered, every total moves up by up to 45 points. Showing the real, honest mid-semester figure is the design working, not a bug.

8 · The services

The only layer that touches the database. Eleven files. Each one loads data, calls a pure function or writes SQL, and returns plain data — never an HTTP response.

The shape every service follows

1. ASSERT OWNERSHIP	<code>assertCourseOwned(courseId, teacherId)</code>
2. VALIDATE THE INPUT	<code>throw badRequest("...")</code> on anything wrong
3. LOAD	<code>select ...</code>
4. THINK	call the pure planner or grading function
5. SAVE	<code>insert / update</code> , inside a transaction if more than one statement
6. RETURN	plain data. No <code>res.json</code> anywhere in this folder.

A service never sees `req` or `res`. It takes ordinary arguments and returns ordinary values, which is what lets `dashboard.service.ts` call three other services directly and `scripts/seed.ts` call them all from a terminal with no server running.

`plan.service.ts` — load, think, save

The largest and most important service. It is the bridge between the pure planner and the database.

The load helpers

Five small functions, each reading one thing. They share a pattern:

```
async function loadTopics(courseId: string, client?: PoolClient): Promise<PlannerTopic[]> {
  const run = client ?? pool;
  const result = await run.query(
    `select id, order_no, title, sessions_needed, min_sessions, priority
     from topic
     where course_id = $1 and status <> 'dropped'
     order by order_no`,
    [courseId],
  );
  return result.rows.map((r) => ({
    id: Number(r.id),
    order_no: r.order_no,
    ...
  }));
}
```

- `client?: PoolClient` and `const run = client ?? pool` — the same function works inside a transaction or outside one. Called with a client it joins the transaction; called without, it borrows from the pool. Without this the code would need two copies of every loader.
- `status <> 'dropped'` — a topic the teacher dropped through the deficit screen must not come back the next time the planner runs.
- `Number(r.id)` — `bigint` comes back from `pg` as a string, and the planner's types say `number`.

generatePlan — the before-the-semester path

```

export async function generatePlan(courseId, teacherId, options: { reset?: boolean } = {}) {
  const course = await assertCourseOwned(courseId, teacherId);

  const conducted = await pool.query(
    `select count(*)::int as n from session where course_id = $1 and status <> 'planned`,
    [courseId],
  );
  const alreadyHappened = conducted.rows[0]?.n ?? 0;

  if (alreadyHappened > 0 && options.reset !== true) {
    throw badRequest(
      `This course already has ${alreadyHappened} class(es) marked conducted or cancelled. ` +
      `Use POST /courses/${courseId}/plan/replan to rebuild the future without touching them, ` +
      `or send { "reset": true } to start the whole plan over ...`,
    );
  }

  return transaction(async (client) => {
    if (options.reset === true) {
      await client.query(`delete from session where course_id = $1`, [courseId]);
    }

    const topics = await loadTopics(courseId, client);
    const holidays = await loadActiveHolidays(courseId, client);
    const makeupDates = await loadMakeupDates(courseId, client);

    // THINK
    const regular = generateSlots(course.start_date, course.end_date, course.class_days, holidays);
    const slots = mergeExtraDates(regular, makeupDates, course.start_date);
    const { sessions, overflow, unusedSlots } = allocateTopics(slots, topics);

    // SAVE
    await writeSessions(client, courseId, sessions, new Set(makeupDates));

    const assessments = await loadAssessments(courseId, client);
    const titles = new Map(topics.map((t) => [t.id, t.title]));
    const moves = validateAssessmentDates(sessions, slots, assessments, titles);
    await applyAssessmentMoves(client, moves);

    return { sessions, overflow, assessments_moved: moves, slots_total: slots.length,
      slots_unused: unusedSlots.length, deficit: calculateDeficit(slots.length, overflow) };
  });
}

```

The guard at the top was added because of a bug the smoke test found. `generate` lays out every topic across **every** slot from the start date. Run mid-semester, it would schedule a fresh class on a date that already holds a conducted one, and it would ignore teaching that has already happened. So it refuses, and the error message names the endpoint to use instead.

`reset: true` is the deliberate escape hatch. It deletes conducted classes and — through on `delete cascade` — their attendance, so it demands an explicit flag rather than being the default.

writeSessions — replacing the future

```

async function writeSessions(client, courseId, sessions: PlannedSession[],
                             makeupDates: Set<string> = new Set()) {
  await client.query(`delete from session where course_id = $1 and status = 'planned'`, [courseId]);

  if (sessions.length === 0) return;

  await client.query(
    `insert into session (course_id, topic_id, date, week_no, part_no, total_parts, kind)
    select $1, t.topic_id, t.date, t.week_no, t.part_no, t.total_parts, t.kind
    from unnest($2::bigint[], $3::date[], $4::int[], $5::int[], $6::int[], $7::text[])
    as t(topic_id, date, week_no, part_no, total_parts, kind)`,
    [courseId, sessions.map((s) => s.topic_id), sessions.map((s) => s.date), ...
     sessions.map((s) => (makeupDates.has(s.date) ? "makeup" : "regular"))],
  );
}

```

- `and status = 'planned'` — this clause is "freeze the past". Conducted and cancelled rows are not in the delete, so they survive every rebuild.
- `if (sessions.length === 0) return` — `unnest` on empty arrays produces no rows, so this is only a small saving, but it also avoids a pointless round trip.
- The whole rebuild is one delete and one insert, inside a transaction. A crash between them leaves the old plan intact.

replanCourse — freeze the past, rebuild the future

Same shape, but it loads the existing sessions, passes everything to `replan()`, and then writes a record of what happened:

```

await client.query(
  `insert into replan_log (course_id, reason, changes) values ($1, $2, $3)`,
  [courseId, reason, JSON.stringify({ changes: result.changes, assessments_moved: moves })],
);

```

`replan_log.changes` is a `jsonb` column, so the whole change list is stored as JSON in one field rather than being spread over a table of its own. It is a log — written once, read back whole for the course report — so a structured column would have been more work for no benefit.

getDeficitOptions and the makeup-date problem

The interesting part of `applyDeficitChoice` is where an agreed makeup date gets stored.

The first attempt wrote makeup dates as `session` rows. That was wrong, and the reason is worth understanding: every replan starts by deleting the planned sessions. So the makeup date would be created, used once, and then silently deleted on the next replan — and since `generateSlots` only ever produces normal class days, it could never be rediscovered. The teacher's decision would quietly evaporate.

Hence `migrations/002_makeup_date.sql`:

```
create table makeup_date (
  id          bigserial primary key,
  course_id   bigint not null references course(id) on delete cascade,
  date        date    not null,
  reason       text,
  created_at  timestampz not null default now(),
  unique (course_id, date)
);
```

An agreed date is stored **once**, permanently, and every later replan reads it back and feeds it to the planner as an extra slot. The tests an agreed makeup date becomes a usable class slot and a makeup date in the past is not offered as a future slot cover both directions.

session.service.ts — marking a class conducted or cancelled

Topic status is derived, never set by hand

```
export async function refreshTopicStatus(client, topicId: string): Promise<void> {
  await client.query(
    `update topic t
      set status = case
        when t.status = 'dropped'           then 'dropped'
        when counts.conducted = 0           then 'pending'
        when counts.conducted >= counts.total then 'completed'
        else 'in_progress'
      end
    from (
      select count(*) filter (where status = 'conducted') as conducted,
             count(*) filter (where status <> 'cancelled') as total
      from session where topic_id = $1
    ) as counts
    where t.id = $1`,
    [topicId],
  );
}
```

A topic's status is **computed from its sessions** every time one changes. Nothing in the app ever sets it to `completed` directly.

If it were set by hand it would drift: cancel a class after marking the topic complete, and the topic is now wrong with nothing to correct it. Deriving it means it cannot be wrong.

- `when t.status = 'dropped' then 'dropped'` — a dropped topic is a deliberate teacher decision and is left alone.

- `count(*) filter (where status <> 'cancelled')` — the denominator excludes cancelled classes, so a topic whose last class was cancelled can still reach `completed`.
- `from (select ...) as counts` — a subquery in the `from` clause of an `update`, so the counts and the update happen in one statement.

The bug this exposed

`refreshTopicStatus` was originally called only from `updateSession` — the `PATCH /sessions/:id` path. But most classes are marked conducted by **taking attendance**, which has its own `update session` statement.

The seed script made it obvious: 19 classes conducted, and the dashboard reported **7% of the syllabus covered**. Only one topic had ever been updated.

The fix was to call it from the attendance path too:

```
const conducted = await client.query(
  `update session set status = 'conducted', updated_at = now()
   where id = $1 and status = 'planned'
   returning topic_id`,
  [sessionId],
);

const topicId = conducted.rows[0]?.topic_id;
if (topicId !== undefined && topicId !== null) {
  await refreshTopicStatus(client, String(topicId));
}
```

After the fix: 57%, which matches 8 of 14 topics.

This is a good example of why the seed script matters. The bug was invisible in unit tests — the pure functions were all correct. It only appeared once a realistic semester of data existed and a number looked wrong.

attendance.service.ts — present by default

One statement marks the whole class

The app sends only the absentees. Everyone else is present.

```
insert into attendance (session_id, student_id, status)
select $1,
       e.student_id,
       case
         when e.student_id = any($3::bigint[]) then 'absent'
         when e.student_id = any($4::bigint[]) then 'leave'
         else 'present'
       end
from enrollment e
where e.course_id = $2
on conflict (session_id, student_id) do update
  set status = excluded.status, marked_at = now()
returning student_id, status
```

- The `select ... from enrollment` drives the insert: one row per enrolled student, automatically. The code never has to know the class list.

- `= any($3::bigint[])` — "is this student id in the absent array?" This is how you test membership of an array in PostgreSQL.
- `case ... else 'present'` — the default, expressed in SQL rather than in a loop.
- `on conflict ... do update` — reopening the screen and fixing a mistake just overwrites.

For 42 students that is one round trip instead of 42.

Cancelled classes and leave are excluded from the percentage

```
with conducted as (
  select id from session where course_id = $1 and status = 'conducted'
)
select ...
  case
    when count(a.id) filter (where a.status <> 'leave') = 0 then null
    else round(100.0 * count(a.id) filter (where a.status = 'present')
              / count(a.id) filter (where a.status <> 'leave'), 2)
  end as percentage
```

Two deliberate exclusions:

- **Only conducted classes count.** A cancelled class was never an opportunity to attend. Counting it would lower every student's percentage because the **teacher** cancelled.
- **Leave is excused.** A `leave` record is removed from the denominator entirely rather than counted as an absence — that is what makes it different from `absent`.

And `case ... then null` guards against dividing by zero before any class has happened. `null`, not `0` — a student with no record yet has no percentage, and showing `0%` would flag them as below threshold on day one.

`100.0` rather than `100` forces floating-point division. In PostgreSQL, `100 / 3` on integers is `33`.

student.service.ts — the reviewed class list

The teacher photographs the printed list, an extractor returns a draft, the teacher corrects it on screen, and **only then** does it reach this service.

The server validates again, even though the screen did

```
function validateRows(value: unknown): ReviewedStudent[] {
  if (!Array.isArray(value) || value.length === 0) {
    throw badRequest("students must be a non-empty array of { roll_no, name }");
  }

  const rows: ReviewedStudent[] = [];
  const seen = new Map<string, number>();

  value.forEach((row, index) => {
    const line = index + 1;
    const rollNo = typeof row?.roll_no === "string" ? row.roll_no.trim() : "";
    const name = typeof row?.name === "string" ? row.name.trim() : "";

    if (rollNo === "") throw badRequest(`Row ${line} has no roll number`);
    if (name === "") throw badRequest(`Row ${line} has no name`);

    const earlier = seen.get(rollNo.toUpperCase());
    if (earlier !== undefined) {
      throw badRequest(`Roll number ${rollNo} appears twice, on rows ${earlier} and ${line}`);
    }
    seen.set(rollNo.toUpperCase(), line);

    rows.push({ roll_no: rollNo, name });
  });

  return rows;
}
```

The review screen highlights these same problems. The checks are repeated here because **the screen is a convenience and the server is the rule** — an old build of the app, a second client, or a replayed request must not get past them.

Duplicates are refused, never merged. Two students with the same roll number means one was misread. Merging them would silently lose a student for the whole semester. The error names both row numbers so the teacher can see which two.

`rollNo.toUpperCase()` for the duplicate check, but the original casing is stored — so `ct-21001` and `CT-21001` are caught as the same student.

The import itself

```
insert into student (teacher_id, roll_no, name)
select $1, r.roll_no, r.name
  from unnest($2::text[], $3::text[]) as r(roll_no, name)
on conflict (teacher_id, roll_no) do update set name = excluded.name
returning id, roll_no, name
```

Then a second statement enrolls them:

```
insert into enrollment (course_id, student_id)
select $1, id from unnest($2::bigint[]) as id
on conflict (course_id, student_id) do nothing
returning student_id
```

Both inside one transaction. Two statements for 42 students, and re-importing the list is safe: existing students are matched by roll number and have their names refreshed, and existing enrollments are left alone.

grading.service.ts — marks in, results out

Marks are validated against the total

```
if (value > totalMarks) {
  throw badRequest(`Row ${line}: obtained (${value}) is more than the total marks (${totalMarks})`);
}
```

A quiz out of 10 cannot have someone scoring 85 — that is a slipped keypad, and catching it at entry is far better than finding it in the final result.

Clearing a mark is allowed and keeps it `null`

```
if (row.obtained !== undefined && row.obtained !== null && row.obtained !== "") {
  value = Number(row.obtained);
  ...
}
```

`value` starts as `null` and only becomes a number if something real was sent. So sending `null` or an empty string sets the mark back to **not entered** — a genuine thing a teacher needs to do after a mistake — and it stays `null` rather than becoming `0`.

A student from another class cannot be marked

```
insert into mark (assessment_id, student_id, obtained, is_absent)
select $1, r.student_id, r.obtained, r.is_absent
from unnest($2::bigint[], $3::numeric[], $4::boolean[])
as r(student_id, obtained, is_absent)
join enrollment e on e.student_id = r.student_id and e.course_id = $5
on conflict (assessment_id, student_id) do update ...
```

The `join enrollment` is the security check, done in SQL. A student id that is not enrolled in this course matches no row and is silently skipped rather than saved. The service reports how many were ignored:

```
return { saved: result.rowCount ?? 0, ignored: studentIds.length - (result.rowCount ?? 0), ... };
```

Lazy seeding of weightage and grade scale

```
export async function getWeightage(courseId: string, teacherId: string) {
  await assertCourseOwned(courseId, teacherId);

  const existing = await pool.query(`select component, percentage from weightage where course_id = $1`, [courseId]);

  if (existing.rowCount === 0) {
    // insert the defaults, then return them
  }
  ...
}
```

Nothing is asked at course setup. The defaults appear the first time the grading screen is opened, already filled in, and the teacher corrects them from there. That is the minimum-input rule applied to a feature that would otherwise need a form nobody wants to fill in on day one.

course.service.ts — settings, topic edits and cloning

replan_required — telling the app when the plan is stale

```
if (body.class_days !== undefined) { ... planAffected = true; }
if (body.start_date !== undefined || body.end_date !== undefined) { ... planAffected = true; }
...
return { ...course, replan_required: planAffected };
```

Changing the dates or the class days invalidates the timetable. Changing the course **name** does not. So the service tracks which kind of change happened and tells the app, which then shows a "your plan needs rebuilding" prompt.

The same flag comes back from `PATCH /topics/:id` when `sessions_needed`, `order_no` or `status` changes — exactly the triggers the spec names for replanning.

Dynamic set clauses, built safely

```
const sets: string[] = [];
const values: unknown[] = [courseId];

const set = (column: string, value: unknown) => {
  values.push(value);
  sets.push(`${column} = ${values.length}`);
};

if (body.name !== undefined) { ...; set("name", name); }
if (body.code !== undefined) set("code", ...);
...

await client.query(
  `update course set ${sets.join(", ")}, updated_at = now() where id = $1 returning ...`,
  values,
);
```

`PATCH` should change only the fields that were sent, so the SQL has to be built at runtime.

The safety comes from what goes where: the **column names** are literals written in this file, and the **values** always become `$2`, `$3` and so on. `values.push` then ``${values.length}`` keeps the numbering in step automatically — push the value first, and its position is the placeholder number.

User input never reaches the query text.

Validating against the database's own rules first

```
if (nextMin > nextNeeded) {
  throw badRequest(`min_sessions (${nextMin}) cannot be more than sessions_needed (${nextNeeded})`);
}
```

The database would refuse this anyway through `topic_min_le_needed_ck`. Checking it here turns

```
error: new row for relation "topic" violates check constraint "topic_min_le_needed_ck"
```

into a sentence a teacher can act on. The constraint stays as the real guarantee; this is just a better error message.

cloneCourse — what is copied and what is not

Copied	Not copied
Topics, reset to pending	Students and enrollments
Weightage	Sessions
Grade scale	Attendance
Class days, threshold, code	Marks

The rule: **the teacher's preparation is copied, the semester's history is not.** Topics and grading setup are the work nobody wants to redo. Students and marks belong to the semester that has ended.

Holidays are a special case — **recalculated** for the new dates rather than copied:

```
const holidays = holidaysBetween(startDate, endDate);
```

Eid moves every year. Copying last year's dates would put a holiday on the wrong day.

dashboard.service.ts — every number in one response

The spec asks for one endpoint that returns everything the dashboard shows, so the app makes one request instead of six and stitching them together.

It works by calling other services:

```
const health = await getScheduleHealth(courseId, teacherId, today);
const attendance = await getAttendanceSummary(courseId, teacherId);
const results = hasStudents ? await getResults(courseId, teacherId) : null;
```

This is only possible because services take plain arguments and return plain data. If they wrote to `res` directly, none of this composition would work.

`hasStudents ? ... : null` — computing results for a course with no students is pointless, and it also avoids the grading engine running over an empty class on every dashboard load.

The warning sentence is assembled here rather than in the app, so every client shows the same wording:

```
warning: health.behind_by_weeks >= 0.5
  ? `You are ${health.behind_by_weeks} week${health.behind_by_weeks === 1 ? "" : "s"} behind your plan.`
  : null
```

The `week${... === 1 ? "" : "s"}` is the singular/plural fix — "1 week", "2 weeks".

report.service.ts — everything except the drawing

The PDF **tool** is not chosen yet. What is finished is everything above it.

```
export type ReportDocument = {
  type: ReportType;
  title: string;
  header: ReportHeader;
  columns: string[];
  rows: (string | number | null)[][];
  summary: { label: string; value: string | number }[];
  notes: string[];
};
```

Every number, every row, every summary line and every footnote is computed — by the same functions the screens use. When a PDF library is picked, the only new code is "draw this object". There is nothing left for it to calculate, so the document cannot disagree with the app.

```
export async function renderReportPdf(_document: ReportDocument): Promise<Buffer> {
  throw new AppError(503,
    "PDF rendering is not configured yet. " +
    "Request the same report without .pdf to get the finished report as JSON.");
}
```

503 Service Unavailable is the honest status: the feature exists and is not available yet. And the message names the workaround, which actually works today — `GET /courses/3/reports/result` returns the complete report as JSON, and the preview screen can be built against it now.

All three reports are done: `result`, `attendance` and `course`. The result sheet names students with missing marks explicitly rather than hiding them, because the total beside them is provisional.

9 · Every endpoint

The complete API. What each one takes, what it gives back, and which service does the work. This is the contract three mobile developers build against, so the names are frozen.

The rules that apply to all of them

- **Every response** has the shape `{ success, data, message }`. No exceptions.
- **Every field name** is `snake_case`, matching the database columns — so there is no mapping layer between SQL and JSON.
- **Every route except** `/health` requires `Authorization: Bearer <token>`.
- **Every** `:id` is scoped to the signed-in teacher. Somebody else's id returns 404.
- **Routes never send errors.** They throw; `errorHandler` answers.

Status codes used

Code	Meaning here
200	Fine
201	Something was created
400	The request was wrong — bad value, missing field, invalid JSON
401	No token, or a token that does not verify
404	Does not exist, or is not yours
409	Would duplicate an existing row
503	A feature whose tool is not chosen yet (PDF, extraction)

Health and identity

Endpoint	What it does
GET <code>/health</code>	<code>{ "status": "ok" }</code> . No database, no token
GET <code>/auth/me</code>	The signed-in teacher. Creates their <code>teacher</code> row on first call

Sign-in itself is not here. It happens in the app, against Supabase.

Courses

Endpoint	Takes	Returns
POST /courses	name, start_date, end_date, class_days, optional code, semester	The new course. Also preloads public holidays
GET /courses	—	The teacher's courses with counts and a has_plan flag
GET /courses/:id	—	One course with its topics and holidays
PATCH /courses/:id	Any of name, code, semester, start_date, end_date, class_days, attendance_threshold	The updated course, plus replan_required
POST /courses/:id/clone	Optional name, semester, start_date, end_date	A new course with the topics and grading setup copied

POST /courses — the minimum to get started

```
{
  "name": "Database Systems",
  "code": "CS-301",
  "semester": "Fall 2026",
  "start_date": "2026-09-01",
  "end_date": "2026-12-18",
  "class_days": ["mon", "wed"]
}
```

Five fields, two of them optional. No students, no weightage, no grade scale — those are asked for later by the features that need them.

Creating the course and preloading its holidays happens in one transaction, so a course can never exist without its holidays.

Topics

Endpoint	Takes	Returns
POST /courses/:id/topics	raw — one topic per line, as a single string	Every topic, numbered in order
PATCH /topics/:id	Any of title, sessions_needed, min_sessions, priority, order_no, status	The updated topic, plus replan_required

POST /courses/:id/topics — one textarea, not a form

```
{
  "raw": "Introduction to Database Systems\nThe Relational Model\nEntity Relationship Modelling\nSQL: Queries"
}
```

The app sends the whole block of text, not an array. The server splits it:

```
const titles = raw
  .split("\n")
  .map((line) => line.trim())
  .filter((line) => line.length > 0);
```

Splitting on the server rather than in the app means all three mobile developers cannot each invent their own splitting rules. Blank lines and stray spaces are handled in one place.

Only the title is required. `sessions_needed` defaults to 1, `min_sessions` to 1, `priority` to `normal` — the teacher corrects them **after** seeing the generated plan, which is when they can actually tell.

This endpoint **replaces** all topics for the course — it deletes then inserts, in a transaction. That is right for the setup screen, where the teacher is editing one textarea. It is not an "add these topics" endpoint.

Holidays

Endpoint	Takes	Returns
GET /courses/:id/holidays	—	Preloaded holidays with their <code>is_active</code> flag
POST /courses/:id/holidays	<code>holidays: [{ id, is_active }]</code>	The updated rows

The teacher unchecks what does not apply rather than typing dates. `src/data/pakHolidays.ts` holds the fixed-date holidays as month and day — not full dates — so one list covers every year:

```
{ month: 9, day: 6, name: "Defence Day" }
```

A year loop builds the actual dates for whatever range the course covers. Lunar holidays like Eid cannot be computed this way and are listed per year.

The planner

Endpoint	Takes	Returns
POST /courses/:id/plan/generate	Optional reset	The whole timetable, overflow, moved assessments, deficit
POST /courses/:id/plan/replan	Optional today, reason	What was frozen, the rebuilt future, and what changed

Endpoint	Takes	Returns
GET /courses/:id/plan/deficit	Optional ?today=	The three resolution options
POST /courses/:id/plan/deficit/apply	option: "drop", "compress" or "extend"	Applies it, then replans
GET /courses/:id/sessions	—	Every class, flat and grouped by week

POST /plan/generate

```
{
  "success": true,
  "data": {
    "sessions": [
      { "date": "2026-09-02", "week_no": 1, "topic_id": 1, "part_no": null, "total_parts": null },
      { "date": "2026-09-07", "week_no": 2, "topic_id": 2, "part_no": 1, "total_parts": 2 },
      { "date": "2026-09-09", "week_no": 2, "topic_id": 2, "part_no": 2, "total_parts": 2 }
    ],
    "overflow": [],
    "assessments_moved": [
      {
        "assessment_id": 3,
        "from": "2026-10-19",
        "to": "2026-11-11",
        "reason": "Quiz 3 moved from 19 Oct to 11 Nov because Normalization will not be completed before 4 Nov"
      }
    ],
    "slots_total": 30,
    "slots_unused": 0,
    "deficit": 0
  },
  "message": null
}
```

Returns 400 if any class has already been conducted, naming `/plan/replan` as the endpoint to use instead.

POST /plan/replan

```
{
  "frozen": 19,
  "sessions": [ ... the rebuilt future ... ],
  "changes": [
    { "kind": "moved", "topic_id": 9, "topic": "File Organisation and Indexing",
      "from": "2026-11-18", "to": "2026-11-23" },
    { "kind": "removed", "topic_id": 14, "topic": "NoSQL and Distributed Databases",
      "from": "2026-12-16" }
  ],
  "assessments_moved": [ ... ],
  "overflow": [ ... ],
  "slots_available": 9,
  "deficit": 2
}
```

`changes` is what the "what moved" screen shows. `frozen` is a count of conducted classes that were not touched.

`today` can be passed in the body to replan as if it were another date — used by the seed script and useful for the demo.

GET /plan/deficit

```
{
  "deficit": 2,
  "drop": {
    "type": "drop",
    "sessions_recovered": 2,
    "covers_deficit": true,
    "topics": [
      { "topic_id": 14, "title": "NoSQL and Distributed Databases", "sessions_freed": 1 },
      { "topic_id": 13, "title": "Recovery and Backup", "sessions_freed": 1 }
    ]
  },
  "compress": {
    "type": "compress",
    "sessions_recovered": 2,
    "covers_deficit": true,
    "topics": [
      { "topic_id": 10, "title": "Query Processing and Optimisation", "from": 2, "to": 1 },
      { "topic_id": 12, "title": "Concurrency Control", "from": 2, "to": 1 }
    ]
  },
  "extend": {
    "type": "extend",
    "sessions_recovered": 2,
    "covers_deficit": true,
    "dates": ["2026-11-19", "2026-11-20"]
  },
  "slots_available": 9,
  "sessions_needed": 11
}
```

`covers_deficit` on each option is the honesty flag — an option may not fully solve the problem, and it says so rather than pretending.

GET /courses/:id/sessions

Returns the plan twice, in two shapes:

```
{
  "sessions": [ ...flat, in date order... ],
  "weeks": [
    { "week_no": 1, "sessions": [ ... ] },
    { "week_no": 2, "sessions": [ ... ] }
  ]
}
```

The grouping is done on the server so every screen that draws a week view does not repeat the same loop. Each session includes `topic_title` from a `left join`, so the app does not need a second request to name the topics.

Sessions and attendance

Endpoint	Takes	Returns
PATCH /sessions/:id	status, and <code>cancel_reason</code> when cancelling	The updated session

Endpoint	Takes	Returns
POST /sessions/:id/attendance	absent: [ids], optional leave: [ids]	Counts of present, absent and leave
GET /sessions/:id/attendance	—	Every student with their status, defaulting to present
GET /courses/:id/attendance/summary	—	Per-student percentages and the below-threshold list

POST /sessions/:id/attendance — only the absentees

```
{ "absent": [12, 40, 41], "leave": [7] }
```

Everyone else enrolled is marked present. For a class of 42 the app sends three or four ids instead of 42.

Submitting attendance also marks the class **conducted**, because that is what actually happened — the teacher held the class and took the roll. And marking it conducted recalculates the topic's status.

`cancel_reason` is **required** when cancelling:

```
if (status === "cancelled" && (typeof cancel_reason !== "string" || cancel_reason.trim() === "")) {
  throw badRequest("cancel_reason is required when cancelling a class");
}
```

Because the course report lists why each class was cancelled, and a blank reason would make that report useless months later.

Students

Endpoint	Takes	Returns
POST /courses/:id/students/extract	storage_path	Parsed rows for review. Saves nothing. 503 until the tool is chosen
POST /courses/:id/students/import	students: [{ roll_no, name }]	The saved students and how many were newly enrolled
GET /courses/:id/students	—	The class list with attendance percentages and a <code>below_threshold</code> flag
DELETE /courses/:id/students/:studentId	—	Removes the enrollment, keeps the student
GET /students/:id	—	One student: full attendance record and every mark

The two-step import, and why it cannot be one step

```

POST /students/extract    -> { rows: [...], saved: false, next_step: "..."}
    |
    REVIEW SCREEN — the teacher fixes what is wrong
    |
POST /students/import     -> saves

```

`extract` deliberately returns `"saved": false` and a `next_step` string. It never writes to the database.

The reason from the spec: printed roll numbers misread easily — `CT-21001` versus `CT-2100I` — and a wrong roll number silently corrupts that student's attendance and result **for the whole semester**, with nothing to reveal the mistake until the final results are wrong. The extraction is a draft; the teacher's confirmation is the source of truth.

The review screen is not optional, and the server enforces its rules again on import.

Assessments, marks and grading

Endpoint	Takes	Returns
POST /courses/:id/assessments	type, title, total_marks, optional date, topic_ids	The new assessment
GET /courses/:id/assessments	—	All of them with <code>marks_entered</code> versus <code>student_count</code>
POST /assessments/:id/marks	marks: [{ student_id, obtained }] or { student_id, is_absent }	How many were saved and how many ignored
GET /assessments/:id/marks	—	Every student in roll order, with whatever marks exist
GET /courses/:id/results	—	Weighted totals, grades and the class summary
GET /courses/:id/weightage	—	The component percentages, seeded on first call
PUT /courses/:id/weightage	The five components	The updated weightage. Must total 100
GET /courses/:id/grade-scale	—	The scale, seeded on first call
PUT /courses/:id/grade-scale	bands: [{ grade, min_percentage }]	The updated scale

type is one of five

`quiz`, `assignment`, `midterm`, `final`, `participation` — the same five as the weightage components, so every assessment maps to exactly one weighted component.

A quiz is simply an assessment of type `quiz`, marked as a single total. There is no per-question marking: for a class of 70 with 10 questions that would be 700 numbers to type, which fails the minimum-input rule outright.

topic_ids is what makes the planner work

```
{ "type": "quiz", "title": "Quiz 3", "date": "2026-10-19", "total_marks": 10, "topic_ids": [7, 8] }
```

Those topic ids are what lets the planner check the quiz is not scheduled before Functional Dependencies and Normalization are taught. Without them the assessment is simply left where it is.

GET /assessments/:id/marks returns everyone

```
{
  "assessment": { "id": 3, "type": "quiz", "title": "Quiz 3", "date": "2026-11-11", "total_marks": 10 },
  "marks": [
    { "student_id": 1, "roll_no": "CT-21001", "name": "Ahmed Khan", "obtained": 7.5, "is_absent": false },
    { "student_id": 2, "roll_no": "CT-21002", "name": "Ali Malik", "obtained": null, "is_absent": false }
  ]
}
```

Every enrolled student appears, including those with no mark yet — otherwise the teacher could not enter one. `obtained: null` means not entered, and it stays `null` all the way to the screen.

Material, dashboard and reports

Endpoint	Takes	Returns
POST /courses/:id/materials	title, storage_path, optional topic_id, mime_type, size_bytes	The saved record
GET /courses/:id/materials	—	The records, flat and grouped into folders by topic
DELETE /materials/:id	—	The removed record and its storage_path
GET /courses/:id/dashboard	Optional ?today=	Every dashboard number
GET /courses/:id/reports /:type	Optional ?today=	The finished report as JSON
GET /courses/:id/reports /:type.pdf	Optional ?today=	The PDF. 503 until the tool is chosen

Files never pass through Express

```
app ---- the file ----> Supabase Storage
app <--- storage path -- Supabase Storage
app ---- the path ----> POST /courses/3/materials
```

The backend only ever stores the path. `DELETE /materials/:id` returns the `storage_path` so the app knows which stored file to delete next — the backend does not delete it either.

GET /courses/:id/dashboard

One request, everything:


```
{
  "course": { ... },
  "today": "2026-11-18",
  "schedule": {
    "planned_up_to_today": 20, "conducted": 19, "cancelled": 2,
    "behind_by_weeks": 0.5, "syllabus_percent": 57, "total_sessions": 30,
    "warning": "You are 0.5 weeks behind your plan."
  },
  "topics": { "total": 14, "completed": 8, "in_progress": 1, "pending": 5, "dropped": 0 },
  "next_session": { "id": 244, "date": "2026-11-18", "week_no": 12, "topic_title": "Entity Relationship Model",
  "attendance": {
    "classes_held": 19, "class_average": 88.18, "student_count": 42,
    "below_threshold_count": 0, "below_threshold": []
  },
  "assessments": {
    "upcoming": [ ... next five ... ],
    "marks_outstanding": [ { "id": 31, "type": "quiz", "title": "Quiz 4", "expected": 42, "entered": 28 },
                          { "id": 32, "type": "final", "title": "Final Examination", "expected": 42, "entered": 28 }
    ],
  "results": { "class_average": 33.25, "highest": 42.19, "lowest": 16.5,
    "grade_distribution": { "F": 42 }, "pass_count": 0, "fail_count": 42,
    "students_with_missing_marks": 42 },
  "last_replan": { "triggered_at": "2026-09-27T00:31:26.137Z", "reason": "2 classes were cancelled during the"
}
}
```

`marks_outstanding` is the useful one for a teacher — it lists exactly which assessments still need marks and how many are missing.

The `.pdf` suffix

```
router.get("/:courseId/reports/:type", requireAuth, async (req, res) => {
  const requested = pathId(req.params, "type");
  const wantsPdf = requested.endsWith(".pdf");
  const type = wantsPdf ? requested.slice(0, -".pdf".length) : requested;

  const document = await buildReport(..., type, today(req.query.today));

  if (!wantsPdf) { ok(res, document); return; }

  const pdf = await renderReportPdf(document);
  res.setHeader("Content-Type", "application/pdf");
  res.setHeader("Content-Disposition", `attachment; filename="${type}-..."`);
  res.send(pdf);
});
```

One route serves both. `:type` arrives as either `result` or `result.pdf`, and the suffix is stripped to decide which.

The JSON form works **today** and returns the complete report — so the preview screen can be built now, and the PDF is a rendering of data that already exists.

Note `res.send(pdf)` rather than `ok(res, pdf)`: a PDF is bytes, not JSON, so it is the one response in the whole API that does not use the envelope. The headers say what it is and give the file a name.

What is not in the contract

Two gaps worth naming rather than quietly adding:

There is no `DELETE /courses/:id`. A teacher cannot delete a course through the API. This is a real gap — the smoke test has to leave its throwaway course behind because of it. It was left out rather than invented, because the endpoint list is frozen and three developers build against it. It is a one-line addition once agreed.

There is no `GET /courses/:id/analysis/topics`. Topic-wise quiz analysis was removed from scope. It required the teacher to tag every quiz question with a topic by hand, which contradicts the minimum-input rule. A quiz is simply an assessment of type `quiz`, marked as a single total.

None of the planner's graded features are affected by that removal. Slot generation, topic allocation, assessment date validation, replanning that freezes the past, and computed deficit resolution are all present and tested.

10 · What is stubbed, and why

Two features are deliberately unfinished. Both are unfinished in the same careful way: everything except the tool is done, so finishing them changes one function each.

The two stubs

Feature	File	Status	Answers
Document extraction	<code>src/services/extraction.service.ts</code>	Interface fixed, no tool chosen	503
PDF rendering	<code>src/services/report.service.ts</code>	Report data complete, no renderer	503

Both are stubbed because the spec says so: **"Tool not decided yet, but the feature is required and fully owned by the app. Leave a stub with a clear interface until the tool is picked."**

Why 503 and not 501 or 500

503 `Service Unavailable` says the feature exists and is temporarily unavailable. That is exactly true. And both messages name a workaround that actually works:

```
"Class list extraction is not configured yet. Use the manual entry path on the
review screen for now."

"PDF rendering is not configured yet. Request the same report without .pdf to
get the finished report as JSON."
```

A 500 would suggest a bug. A silent empty response would have the mobile team debugging their own code.

The extraction stub

What is fixed is the **shape of the answer**:

```
export type ExtractedStudent = {
  roll_no: string | null;
  name: string | null;
  confidence: number;           // 0-1, so the review screen can highlight doubt
};

export type ExtractionResult<T> = {
  rows: T[];
  page_count: number;
  source: string;              // which tool read it
};
```

`roll_no` and `name` are both nullable, because a photo of a printed list genuinely does produce rows where one is unreadable. The review screen flags those. A type that promised `string` would force the eventual implementation to invent a value.

`confidence` exists so the screen can highlight the rows most likely to be wrong — which is the whole point of the review step.

Two rules will survive whichever tool is chosen, and they are written into the file:

- **Nothing extracted is ever written straight to the database.** It goes to the review screen first. A misread roll number silently corrupts that student's attendance and result for the entire semester.
- **The file never travels through Express.** The app uploads to Supabase Storage and sends the path, which is what these functions take.

The PDF stub

The part that is **not** a stub is everything above the renderer. `buildReport` returns a complete document:

```
export type ReportDocument = {
  type: ReportType;
  title: string;
  header: ReportHeader;           // course, teacher, date, signature line
  columns: string[];             // column headings, in order
  rows: (string | number | null)[][]; // one array per row
  summary: { label: string; value: string | number }[];
  notes: string[];               // footnotes, e.g. missing marks
};
```

All three reports are finished — `result`, `attendance` and `course` — and they can be fetched today as JSON.

So `renderReportPdf` has no arithmetic to do:

```
export async function renderReportPdf(_document: ReportDocument): Promise<Buffer> {
  throw new AppError(503, "PDF rendering is not configured yet. ...");
}
```

When a library is picked, it becomes "draw the header, draw a table from `columns` and `rows`, draw the summary, draw the notes, draw a signature line". It does no calculation, touches no database, and makes no formatting decisions beyond layout — every value it prints is already in the object it is handed.

That is the spec's rule: **"Computation is a pure function, kept separate from PDF rendering. Never calculate inside the PDF code."**

11 · Seeding and smoke testing

Two scripts that are not part of the server but are how you know it works.

`scripts/seed.ts` — a realistic full semester

```
npm run seed
```

It builds a complete, believable semester: **Database Systems (CS-301), Fall 2026**, 1 September to 18 December, Mondays and Wednesdays, with

- 14 real Database Systems topics needing 30 classes across 30 available slots — deliberately tight, so disruptions bite
- 42 students with Pakistani names and roll numbers CT-21001 upward
- 8 assessments: four quizzes, two assignments, a midterm and a final
- 19 classes conducted with attendance, 2 cancelled
- 280 marks entered, with Quiz 4 **deliberately** left half-done
- 4 material records
- a replan triggered by the cancellations

It runs through the real services

```
import { generatePlan, replanCourse, getDeficitOptions } from "../src/services/plan.service.ts";
import { importStudents } from "../src/services/student.service.ts";
import { submitAttendance } from "../src/services/attendance.service.ts";
import { createAssessment, saveMarks, getResults } from "../src/services/grading.service.ts";
```

It does not insert rows directly — it calls the same functions the endpoints call. So running it is an end-to-end test: if the planner, the attendance rules or the grade computation were broken, the script would fail or print wrong numbers.

That is exactly how the topic-status bug was found. The pure functions were all correct and all 55 tests passed, but the seed printed **"syllabus covered 7%"** after 19 conducted classes. Chapter 8 has the fix.

The data is deterministic

```
function makeRandom(seed: number) {
  let state = seed;
  return () => {
    state = (state * 1103515245 + 12345) & 0x7fffffff;
    return state / 0x7fffffff;
  };
}

const random = makeRandom(20260927);
```

A hand-written pseudo-random generator rather than `Math.random()`, so **every run produces the same demo**. The same students are absent on the same days and score the same marks.

This matters for a demo: you can rehearse it, and the numbers on the screen are the numbers you practised with. With `Math.random()` the class average would be different every time.

This script is the only place in the repo allowed to contain invented data. CLAUDE.md: **"No mock or placeholder data in application code. Seed data goes in a seed script."** Nothing in `src/` contains a fake student or a fake mark.

It is safe to run twice

```
const removed = await pool.query(
  `delete from course where teacher_id = $1 and code = $2 returning id`,
  [teacherId, COURSE.code],
);
```

It deletes any previous CS-301 for this teacher first, so repeat runs do not pile up courses. on `delete cascade` removes everything belonging to that course with it.

`scripts/create-demo-user.ts`

The seed needs a teacher, and a teacher must be a real Supabase user because of the foreign key to `auth.users`. This script creates one through the Supabase Admin API:

```
const response = await fetch(`${SUPABASE_URL}/auth/v1/admin/users`, {
  method: "POST",
  headers: {
    apikey: SERVICE_ROLE_KEY,
    Authorization: `Bearer ${SERVICE_ROLE_KEY}`,
    "Content-Type": "application/json",
  },
  body: JSON.stringify({ email, password, email_confirm: true, ... }),
});
```

It is a **script**, not an endpoint, and that is deliberate: we do not build authentication. Creating a user is an administrative job done once from a terminal. The service role key is what makes the call allowed, and it stays on the server.

The demo login it creates:

```
email      demo.teacher@ustaadassist.test
password   UstaadAssist#Demo2026
```

`email_confirm: true` skips the confirmation email, which is right for a test account on a development project.

scripts/smoke.sh — is every route actually wired up

```
npm run smoke 8          # 8 is the seeded course id
```

It signs in, then calls all 33 endpoints once and checks the status code:

```
ok    200 GET    /health
ok    200 GET    /auth/me
ok    401 GET    /auth/me with no token
ok    404 GET    /nothing-here
ok    404 GET    /courses/999999
ok    400 PUT    /courses/8/weightage
ok    400 POST   /courses/8/materials
...
ok    503 GET    /courses/8/reports/result.pdf
ok    503 POST   /courses/8/students/extract

33 passed, 0 failed
```

It is a different kind of test from `npm test`

	<code>npm test</code>	<code>npm run smoke</code>
Checks	that the answers are right	that every route is reachable
Needs	nothing	the server and the database
Speed	300ms	a few seconds
Catches	wrong arithmetic, wrong dates	unmounted routers, renamed routes, a service that throws on empty data

The unit tests cannot catch a router that was never added to `index.ts` — every function it calls is perfectly correct, it is just unreachable. The smoke test cannot catch a wrong week number. Both are needed.

The failures it deliberately checks

The interesting half of the script is the expected **failures**:

```
check 404 GET    /courses/999999          # somebody else's course
check 400 PUT    "/courses/$COURSE/weightage" '{"quiz":10,"final":10}' # does not total 100
check 400 POST   "/courses/$COURSE/materials" '{"title": broken}'      # not valid JSON
check 503 GET    "/courses/$COURSE/reports/result.pdf"                  # stub
```

A test suite that only checks the happy path would pass on an API that accepted a weightage of 20% and let anyone read anyone's course.

Two bugs it found

Both are in this guide already, and both are the kind unit tests structurally cannot find:

- `plan/generate mid-semester` scheduled a second class onto dates that already held conducted classes. Chapter 8 has the guard that now refuses it.
- **A malformed JSON body returned 500** `Something went wrong`. Chapter 5 has the fix: it is now a 400 naming the problem.

The full verification, in order

```
npm run typecheck      # 0 errors
npm test               # 55 passed
npm run dev            # in one terminal
npm run seed           # in another
npm run smoke 8        # 33 passed
```

Current state: type-check clean, 55 unit tests passing, 33 endpoints answering correctly against a real PostgreSQL database and real Supabase tokens.

12 · Quick reference

The things worth having at hand — for a viva, or for picking the project back up in three months.

The five questions most likely to be asked

"Why is this not just a CRUD app?"

Because the central feature is a **computation**, not storage. The planner takes dates, class days, holidays and topics and derives a timetable that nobody typed. Then it maintains it: cancel a class and it rebuilds the future while leaving conducted classes untouched, and when the semester runs short it computes three different resolutions and explains the trade-offs. None of that is Create, Read, Update or Delete.

"Why no ORM?"

Two reasons. A defensible one: an ORM would have generated the SQL, and the interesting queries here are ones it would generate badly — inserting 42 attendance rows in one statement with `unnest`, several counts in one pass with `count(*) filter`, an upsert with `on conflict`. And an honest one: writing the SQL means understanding it, and "the library did it" is not an answer.

"Why is the planner pure?"

So it can be tested, so it cannot surprise you, and so it can be explained. 55 tests run in 300 milliseconds with no database. `today` is an argument rather than read from the clock, which is what lets a test ask "pretend it is 18 November" and what lets the demo show a whole semester in September.

"How is authentication handled?"

It is not built. Supabase Auth issues an ES256-signed JWT inside the app. The backend verifies the signature against Supabase's **public** key, fetched once from the JWKS endpoint and cached. It never contacts Supabase to check a token and never holds a key that could create one. Then every query filters on the user id from that verified token.

"What stops me reading another teacher's course?"

Every query that takes an id from the URL also filters on `teacher_id` from the token — never from the request body. `src/services/ownership.ts` centralises it. A course that exists but is not yours returns 404, not 403, because 403 would confirm the id is real.

The five rules that must not be broken

Rule	Why	Breaking it looks like
The planner never queries the database	Testability, purity	A <code>pool.query</code> inside <code>src/planner/</code>
<code>mark.obtained</code> stays nullable with no default	A missing mark is not a zero	<code>not null default 0</code> "to simplify the code"
Conducted sessions are never rewritten	Freeze the past	Deleting sessions without <code>and status = 'planned'</code>
The owner comes from the token, never the body	Anyone can put anything in a body	<code>teacher_id: req.body.teacher_id</code>

Rule	Why	Breaking it looks like
Round once, at the end	Early rounding moves grade boundaries	<code>round2()</code> called inside the component loop

The numbers

Tables	16 across 2 migrations
Endpoints	33
Runtime dependencies	6
Pure planner files	6, with no database access
Tests	55, running in about 300ms
Lines of SQL written by a library	0

Where to look for what

If you need to...	Go to
Change how the timetable is built	<code>src/planner/allocate.ts</code>
Change which dates are usable	<code>src/planner/slots.ts</code>
Change how a quiz gets moved	<code>src/planner/assessments.ts</code>
Change what happens when time runs out	<code>src/planner/deficit.ts</code>
Change how grades are computed	<code>src/grading/compute.ts</code>
Add an endpoint	the matching file in <code>src/routes/</code> , then a service
Change the error format	<code>src/middleware/error.ts</code> , and nowhere else
Change the response envelope	<code>src/http.ts</code> , and nowhere else
Add a table	a new numbered file in <code>migrations/</code>
Change the demo	<code>scripts/seed.ts</code>

Two things left open

There is no `DELETE /courses/:id`. A teacher cannot delete a course. It was left out rather than invented, because the endpoint list is frozen and three developers build against it. It is a one-line addition once agreed.

The two stubs need their tools chosen. Document extraction and PDF rendering. Everything except the tool is finished in both cases, and the interfaces are fixed, so neither will ripple outward.

Finally: the order things were built in

Worth knowing, because it explains why the code looks the way it does.

The planner was built **first**, before any route that uses it — because `session` is the table everything else hangs off. Attendance is per session. Topic progress is derived from sessions. Every report counts sessions. Once the planner existed and was tested, attendance, marks, the dashboard and the reports were each a short service on top of data that was already correct.

Building it the other way round — CRUD first, planner last — would have meant retrofitting the interesting part into a schema shaped by the boring parts.